

LAB

Mengevaluasi kinerja agen AI menggunakan MLflow

Daftar Isi

Pendahuluan	2
Persyaratan perangkat lunak	2
Tujuan.....	2
Langkah-langkah laboratorium	2
Perkiraan durasi untuk diselesaikan	2
Skenario.....	2
Informasi latar belakang	2
Tantangan	3
Solusi.....	3
Membuat akun GitHub	3
Ringkasan.....	3
Instruksi	3
Membuat akun Replicate.....	9
Ringkasan.....	9
Instruksi	9
Mendaftar ke Google Colab	15
Ikhtisar	15
Instruksi	15
Menginisialisasi agen AI dan alat	19
Ikhtisar	19
Instruksi	19
Mengevaluasi trajektori dan respons akhir dari agen AI	29
Ikhtisar	29
Instruksi	29
Kesimpulan.....	32

Pengantar

Di lab ini, kamu akan berperan sebagai spesialis AI untuk mengevaluasi versi pilot dari agen AI dengan menggunakan MLflow.

Persyaratan perangkat lunak

Untuk menyelesaikan lab ini, kamu perlu akses ke akun Replicate, yang memungkinkan kamu menggunakan model kecerdasan buatan (AI) untuk menjalankan tugas. Kamu juga perlu antarmuka pemrograman aplikasi (application programming interface - API) token dari Replicate, yang berfungsi sebagai kunci aman untuk menghubungkan lingkungan Colab kamu supaya lab ini bisa berjalan lancar.

Pemahaman dasar tentang Python akan membantumu lebih mudah mengikuti cara kerja agen dan penggunaan alatnya.

Tujuan

Setelah menyelesaikan lab ini, kamu diharapkan bisa:

- Mengevaluasi kinerja agen AI menggunakan MLflow

Langkah-langkah Lab

Lab ini mengharuskan kamu untuk menyelesaikan prosedur-prosedur berikut:

- Membuat akun GitHub
- Membuat akun Replicate
- Mendaftar ke Google Colab
- Menginisialisasi agen AI dan alat
- Mengevaluasi trajektori dan respons akhir dari agen AI

Perkiraan durasi untuk menyelesaikan

30 menit

Skenario

Informasi latar belakang

TechMart, sebuah platform e-commerce yang sedang berkembang pesat, tengah mentransformasi pengalaman belanjanya dengan agen AI yang dibangun di atas IBM Granite. Agen ini menggantikan katalog statis dengan memberikan rekomendasi real-time, menjawab pertanyaan, dan mempersonalisasi perjalanan belanja pelanggan.

Kamu adalah bagian dari tim evaluasi AI yang baru dibentuk di TechMart, dan berperan sebagai spesialis AI untuk proyek ini. Peranmu adalah menilai kinerja agen dengan mengevaluasi rekomendasi produk, akurasi respons, dan pengalaman pengguna secara keseluruhan.

Tantangan

Pengguna TechMart sering meninggalkan situs ketika agen AI tidak mampu memberikan rekomendasi produk yang relevan atau tidak efektif membantu dalam navigasi katalog. Selain itu, detail produk, harga, atau ketersediaan stok yang hilang atau tidak konsisten bisa mengurangi kepercayaan pengguna.

Solusi

Untuk mendukung hal ini, timmu menggunakan MLflow untuk melacak, mengevaluasi, dan memvisualisasikan kinerja agen. Di lab ini, kamu akan menentukan apakah agen AI sudah siap untuk disebarluaskan secara penuh.

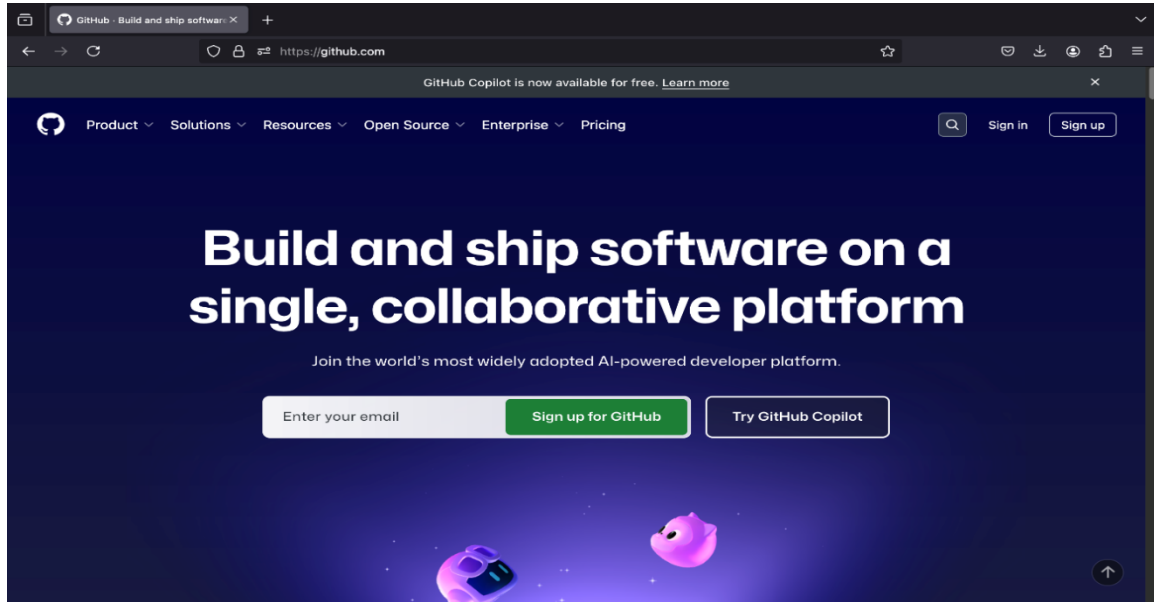
Membuat akun GitHub

Gambaran umum

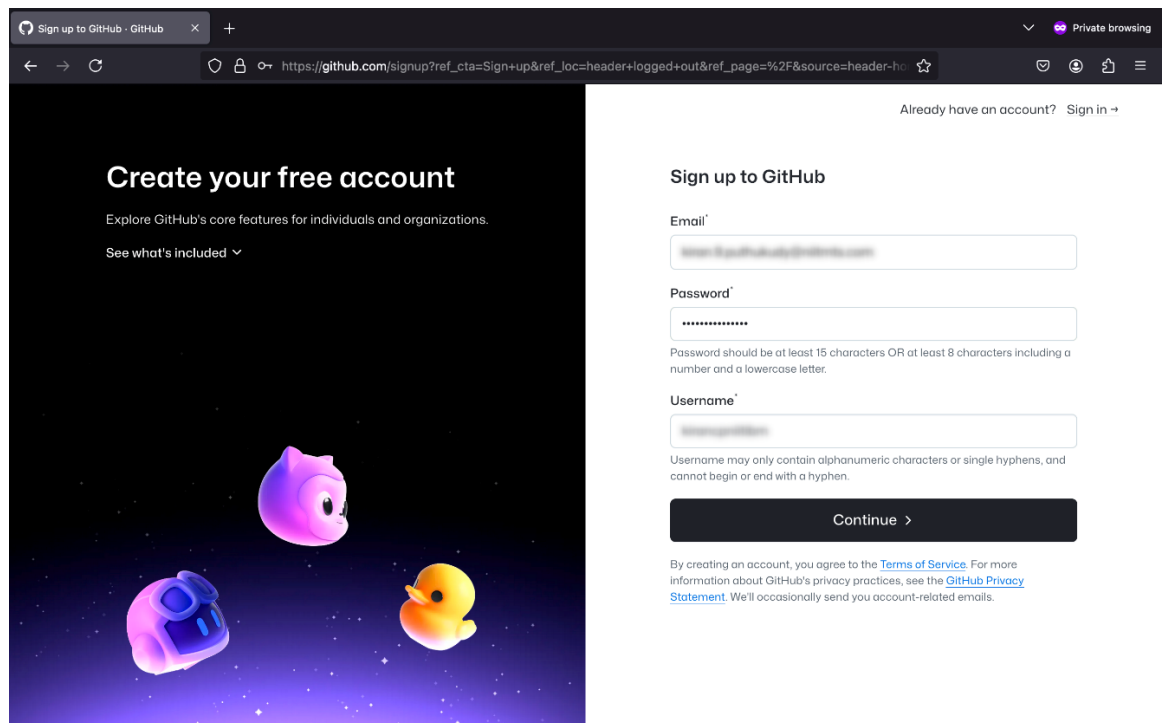
Dalam prosedur ini, kamu akan menyiapkan akun GitHub. GitHub adalah platform yang membantu pengembang menyimpan, mengelola, dan berbagi kode, sekaligus mendukung kolaborasi melalui alat, seperti kontrol versi, pelacakan bug, dan manajemen tugas. Menyiapkan akun GitHub memberimu akses ke platform Replicate, yang diperlukan agar lab ini bisa diselesaikan dengan efisien.

Instruksi

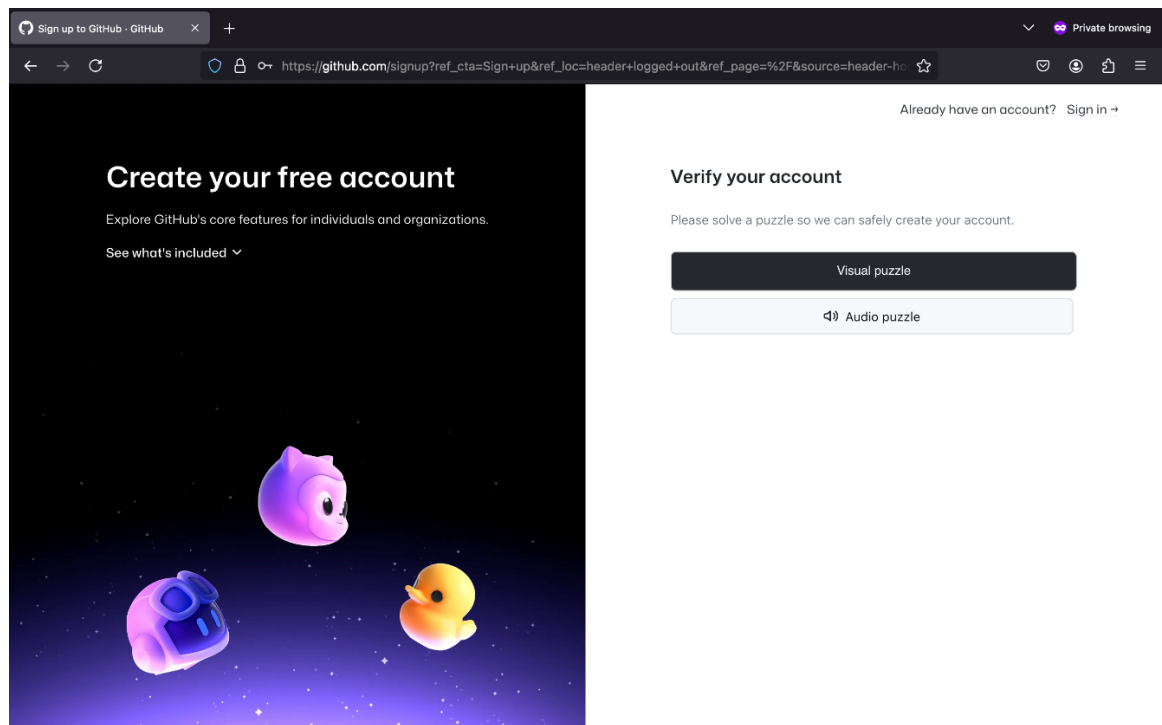
1. Untuk membuat akun GitHub, buka situs web [GitHub](https://github.com).
2. Pilih tombol **Sign up for GitHub** (Daftar GitHub) seperti yang ditunjukkan pada screenshot berikut.



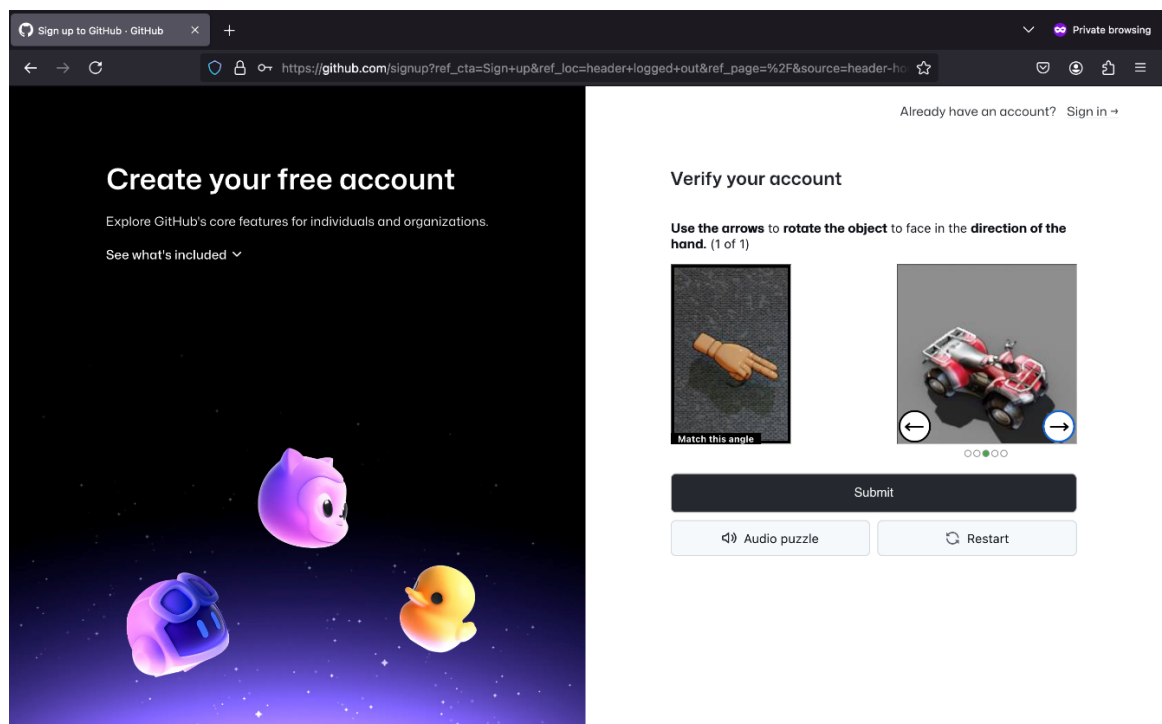
3. Ketik detail kamu di bidang **Email** (Email), **Password** (Kata Sandi), dan **Username** (Nama Pengguna). Lalu, pilih **Continue** (Lanjutkan). Screenshot berikut menunjukkan halaman "Sign up to GitHub" (Daftar ke GitHub).



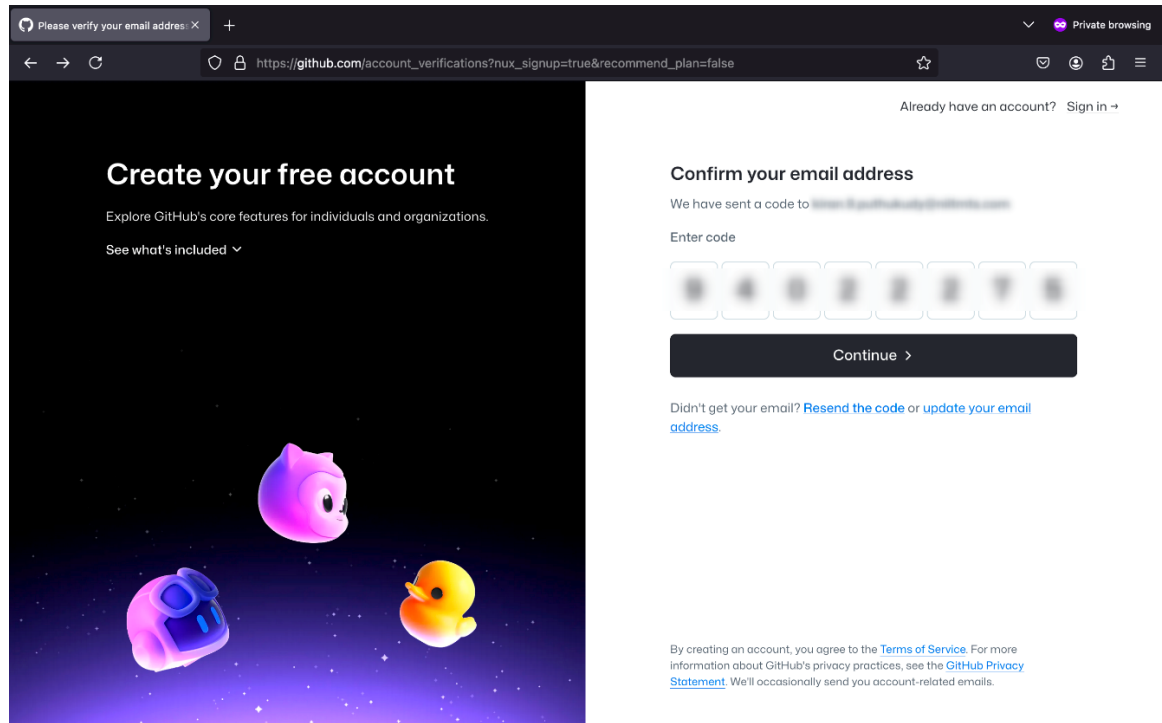
4. Untuk memverifikasi akunmu, pilih tombol **Visual puzzle** (Teka-teki Visual).



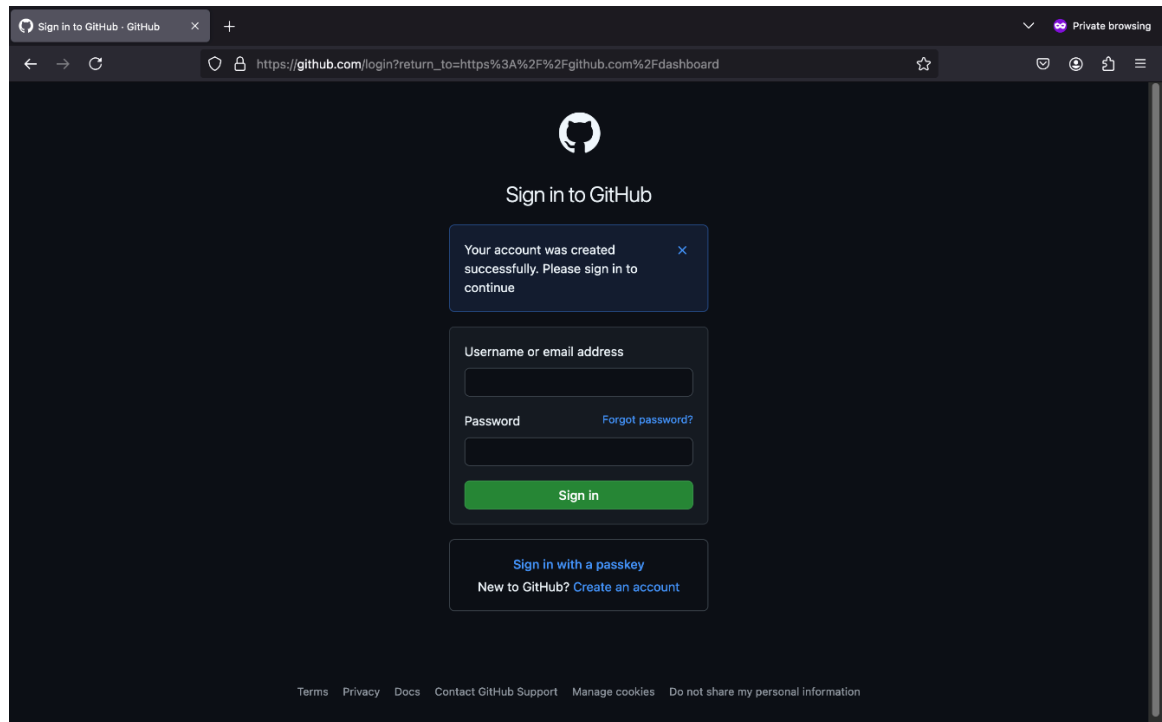
5. Pecahkan teka-teki visual, lalu pilih **Submit** (Kirim).



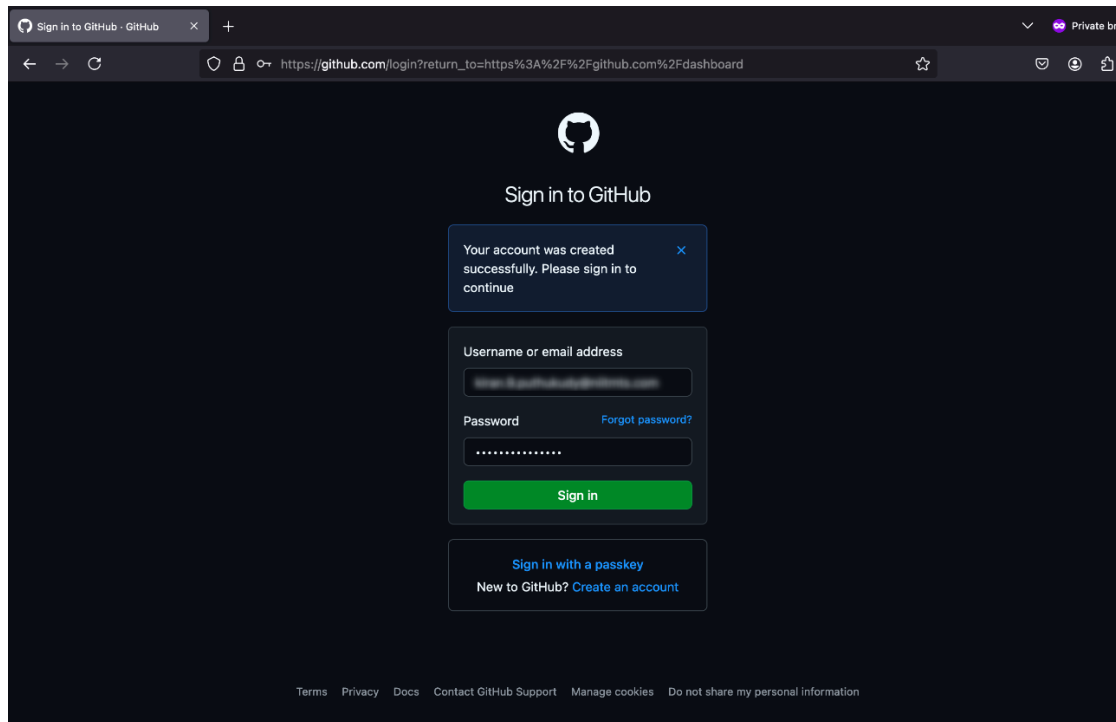
6. Untuk mengonfirmasi email, masukkan kode konfirmasi yang dikirim ke email terdaftar kamu pada bidang **Enter code** (Masukkan kode), lalu pilih **Continue** (Lanjutkan).



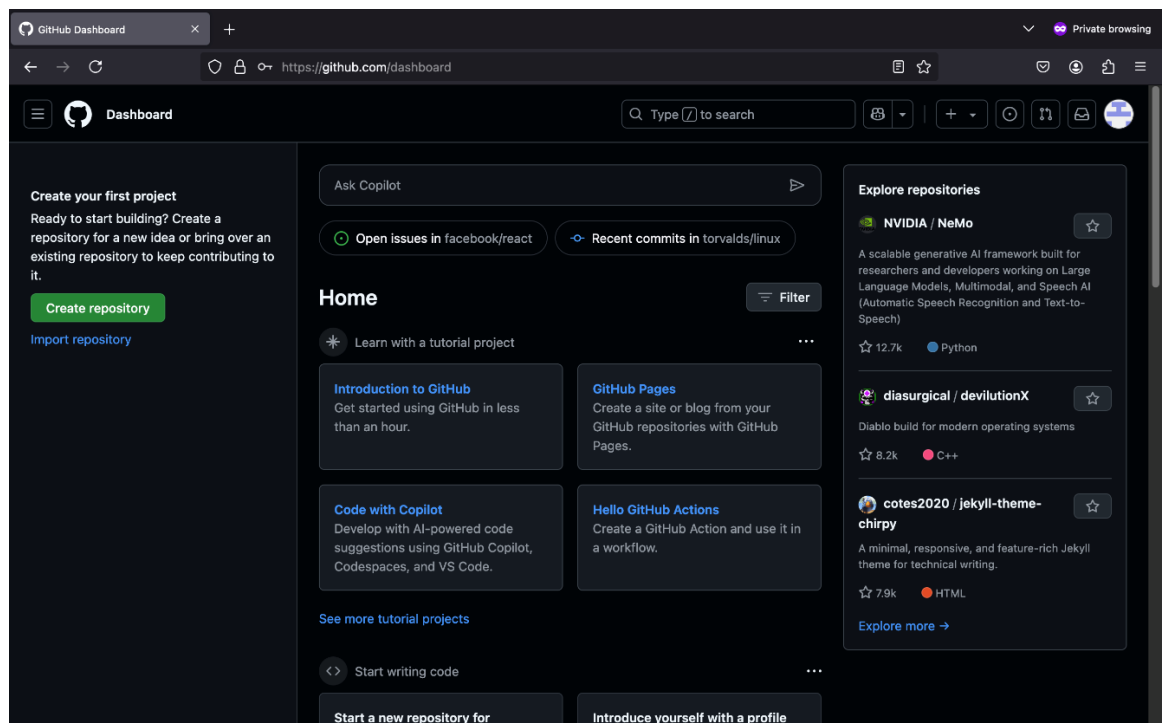
Setelah akun GitHub kamu berhasil dibuat, akan muncul pesan konfirmasi seperti yang ditunjukkan pada screenshot berikut.



7. Untuk masuk ke akunmu, masukkan kredensial di bidang **Username or email address** (Nama pengguna atau alamat email) dan **Password** (Kata sandi), lalu pilih **Sign in** (Masuk). Screenshot berikut memperlihatkan halaman masuk GitHub yang berisi bidang untuk memasukkan nama pengguna dan kata sandi



Setelah kamu masuk, dasbor GitHub akan muncul seperti yang ditunjukkan pada screenshot berikut.



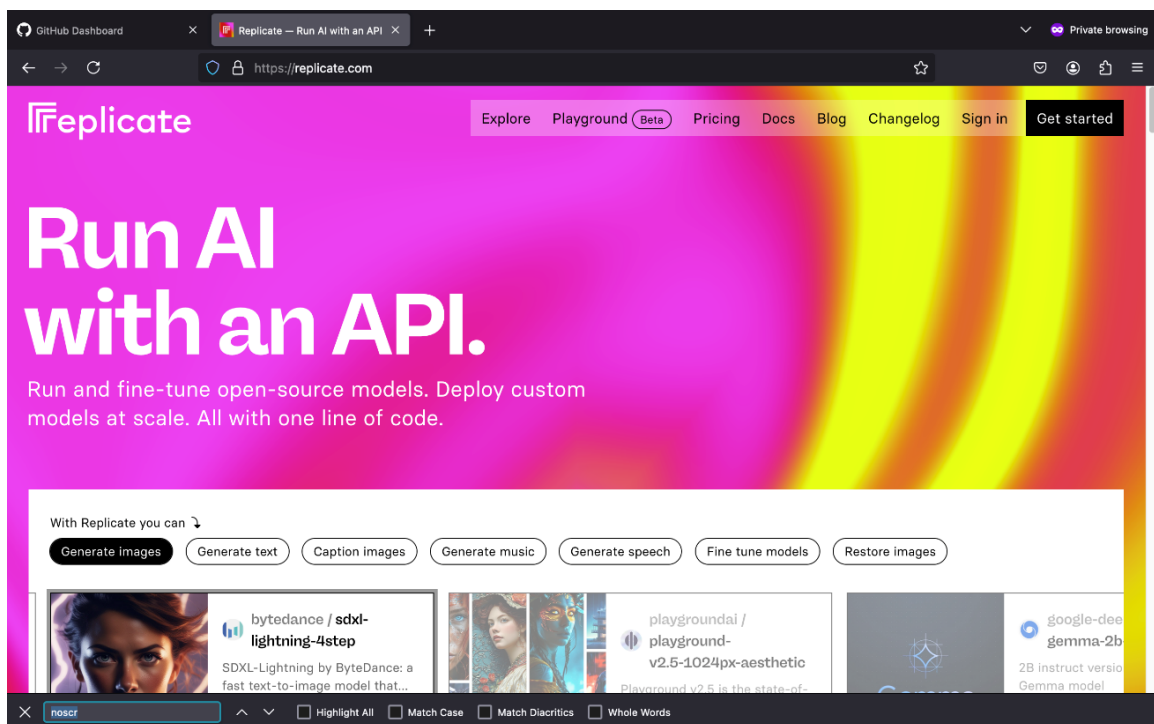
Membuat akun Replicate

Gambaran umum

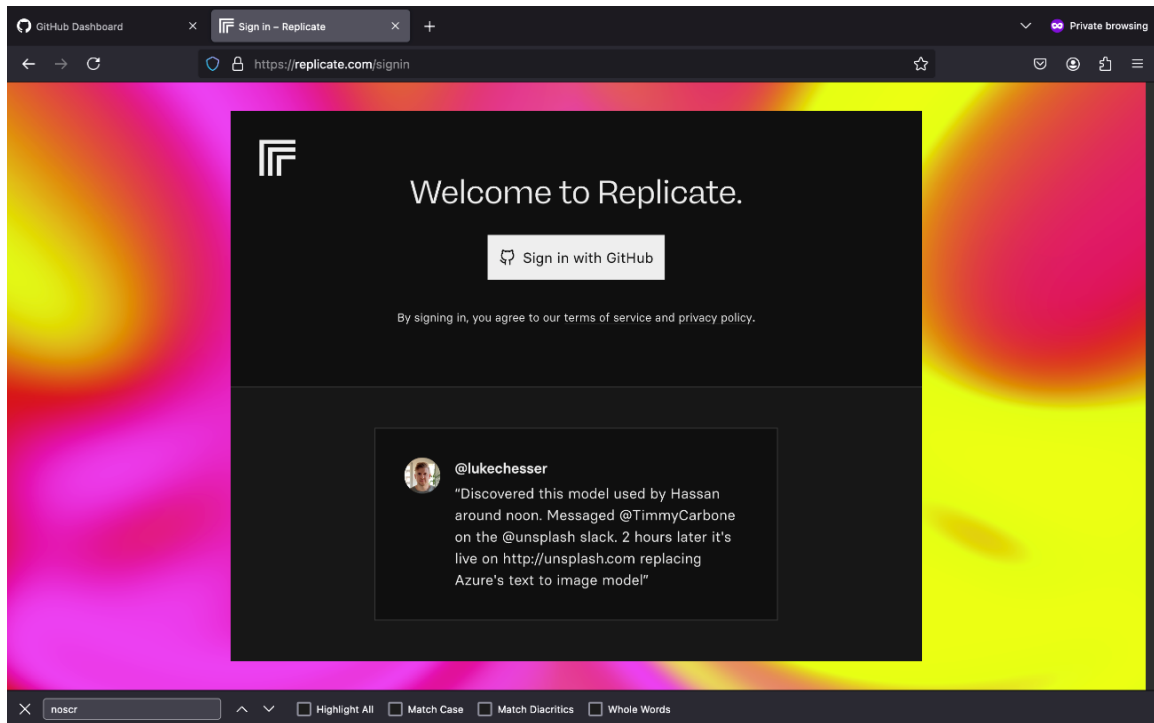
Pada prosedur ini, kamu akan menggunakan akun GitHub untuk mendaftar akun Replicate. Replicate adalah platform berbasis cloud yang memungkinkan kamu menggunakan model AI seperti IBM Granite tanpa perlu perangkat keras canggih. Sebagai bagian dari prosedur ini, kamu akan membuat token Replicate. Token adalah kunci akses aman yang memungkinkan lingkungan lab melakukan autentikasi dengan Replicate dan menjalankan model dari akun kamu di Google Colab.

Instruksi

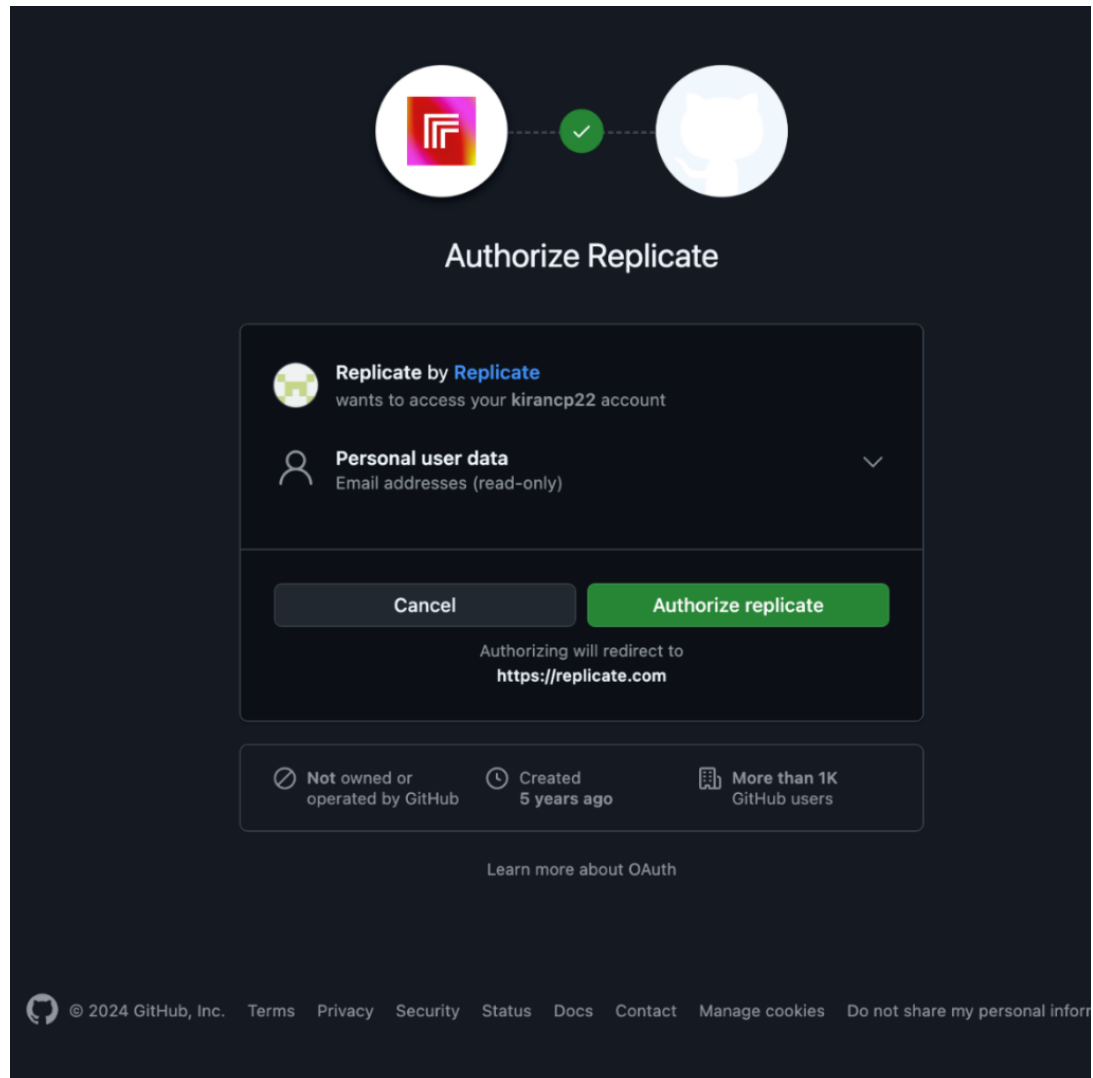
1. Kunjungi situs web [Replicate](https://replicate.com).
2. Pilih **Get started** (Mulai), seperti yang ditunjukkan pada screenshot berikut.



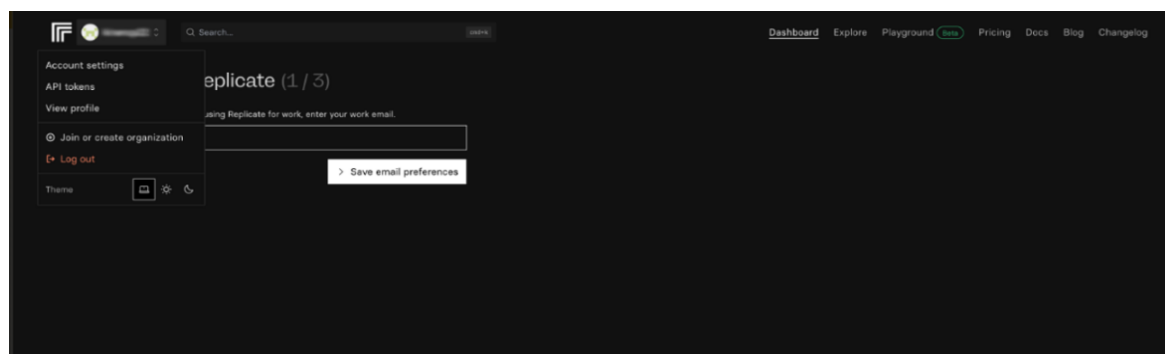
- Pilih tombol **Sign up for GitHub** (Daftar GitHub), seperti yang ditunjukkan pada screenshot berikut.



- Pilih **Authorize replicate** untuk melanjutkan, seperti yang ditunjukkan pada screenshot berikut.

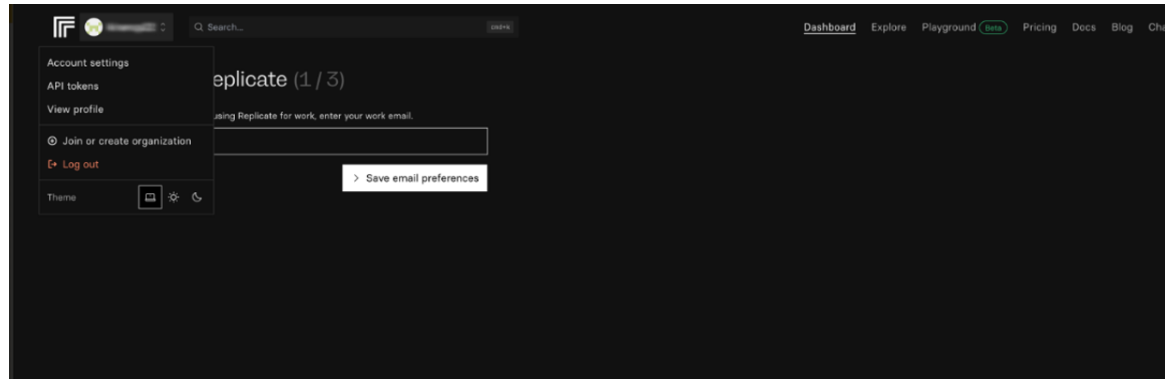


- Setelah akun Replicate kamu dibuat, dasbor Replicate akan muncul, seperti yang ditunjukkan pada screenshot berikut. Untuk membuat token Replicate, pilih opsi **Account settings** (Pengaturan akun) dari menu navigasi.

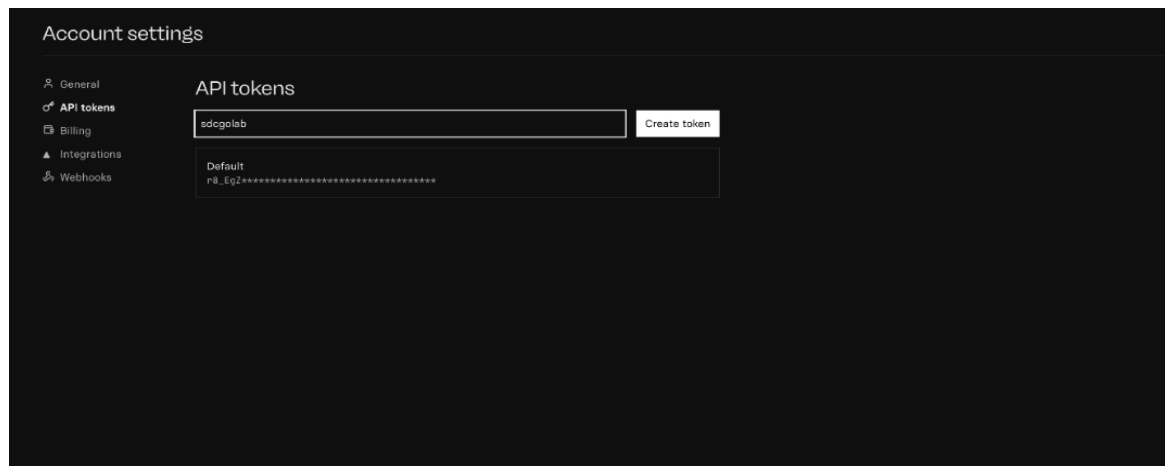


- Pilih **token API** pada panel **Account settings** (Pengaturan akun).

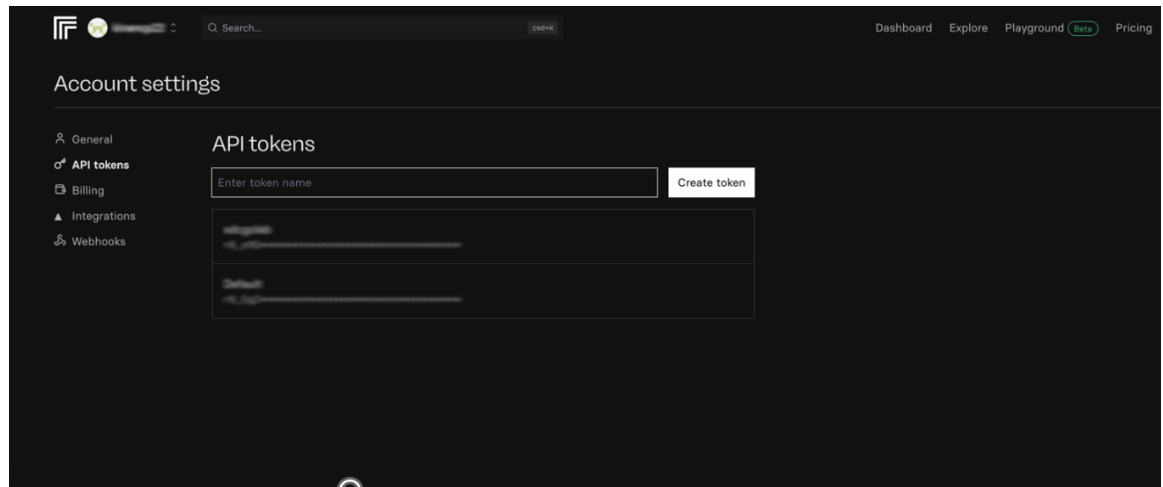
Screenshot berikut memperlihatkan dasbor Replicate.



- Masukkan nama untuk token di bidang **API tokens** (Token API), lalu pilih **Create token** (Buat token) seperti yang ditunjukkan pada screenshot berikut.

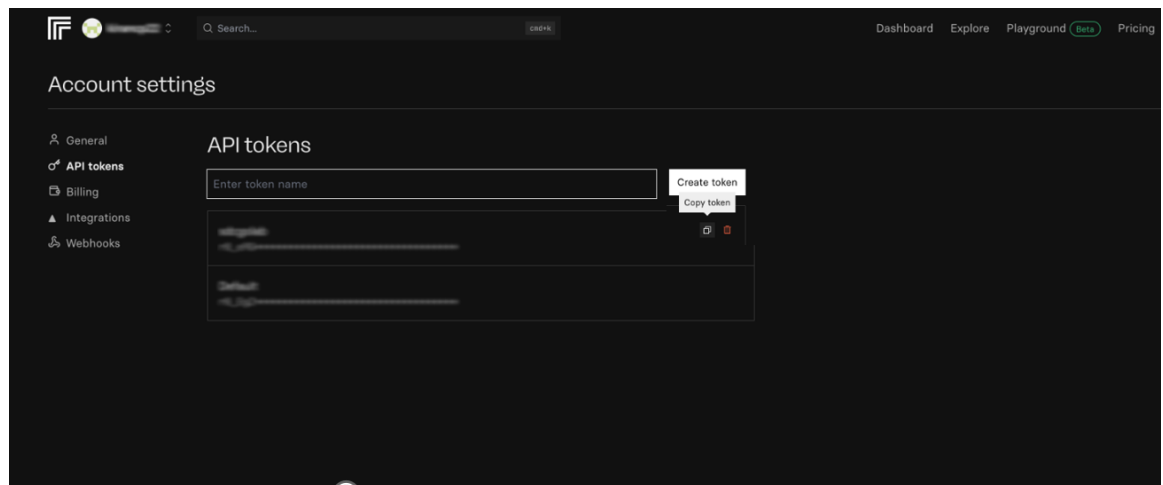


Setelah token API kamu dibuat, token tersebut akan ditampilkan di halaman Account settings (Pengaturan akun) seperti yang ditunjukkan pada screenshot berikut.

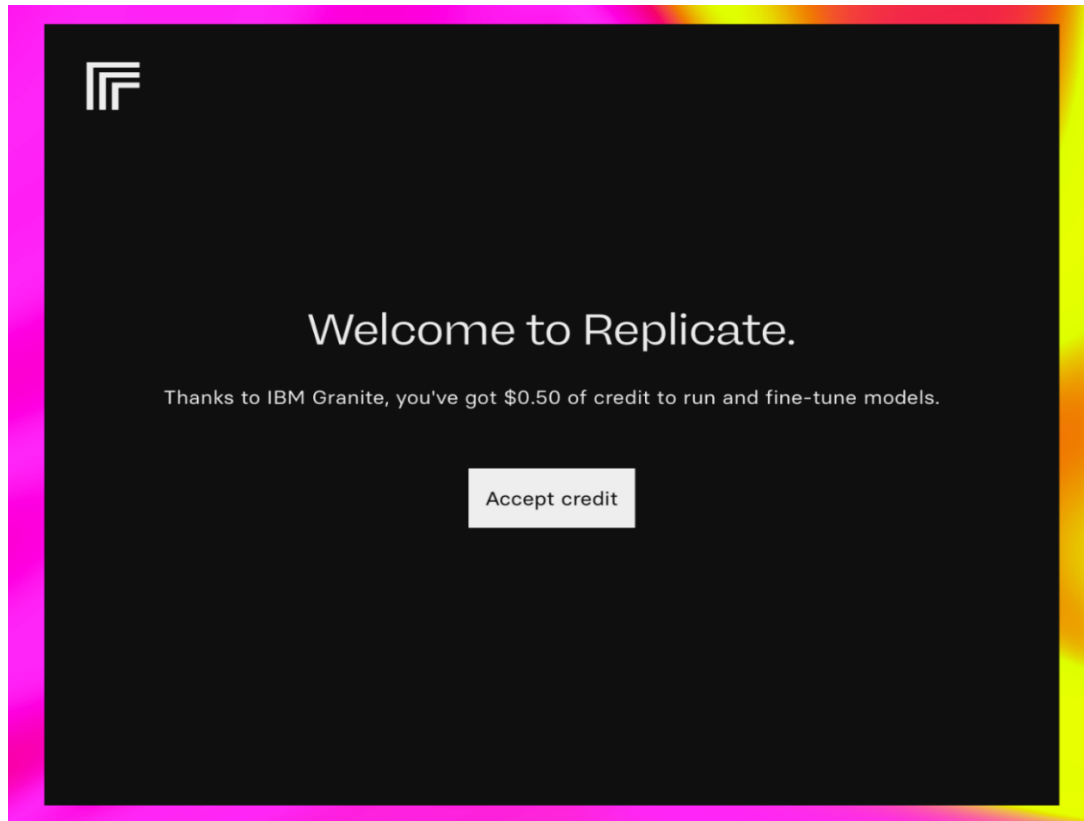


8. Untuk menyalin Replicate API, pilih ikon **Copy token** (Salin token) seperti yang ditunjukkan pada screenshot berikut.

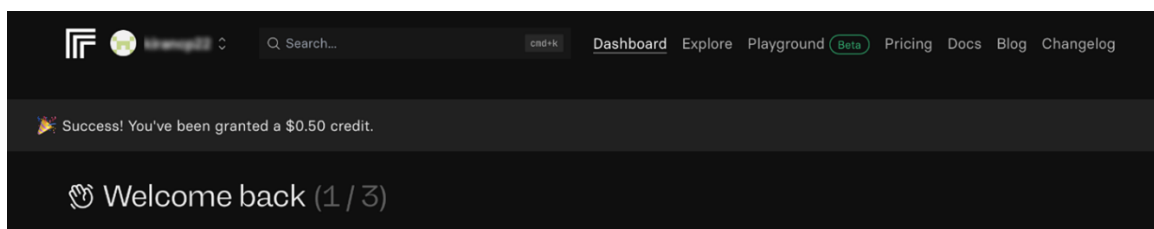
Catatan: Simpan token Replicate karena kamu akan membutuhkannya untuk autentikasi dengan Replicate API saat menjalankan kode di lingkungan Google Colab nanti dalam lab ini.



9. Untuk menjalankan lab tanpa gangguan, kamu memerlukan beberapa kredit Replicate. Buka tautan [Replicate invite](#) (undangan Replicate) untuk mengklaim kredit gratis senilai \$0,50.
10. Untuk mengklaim jumlah tersebut, pilih **Accept credit** (Terima kredit).



Pesan konfirmasi akan muncul yang menunjukkan bahwa kredit \$0,50 telah ditambahkan ke akun Replicate kamu, seperti yang ditunjukkan pada screenshot berikut.



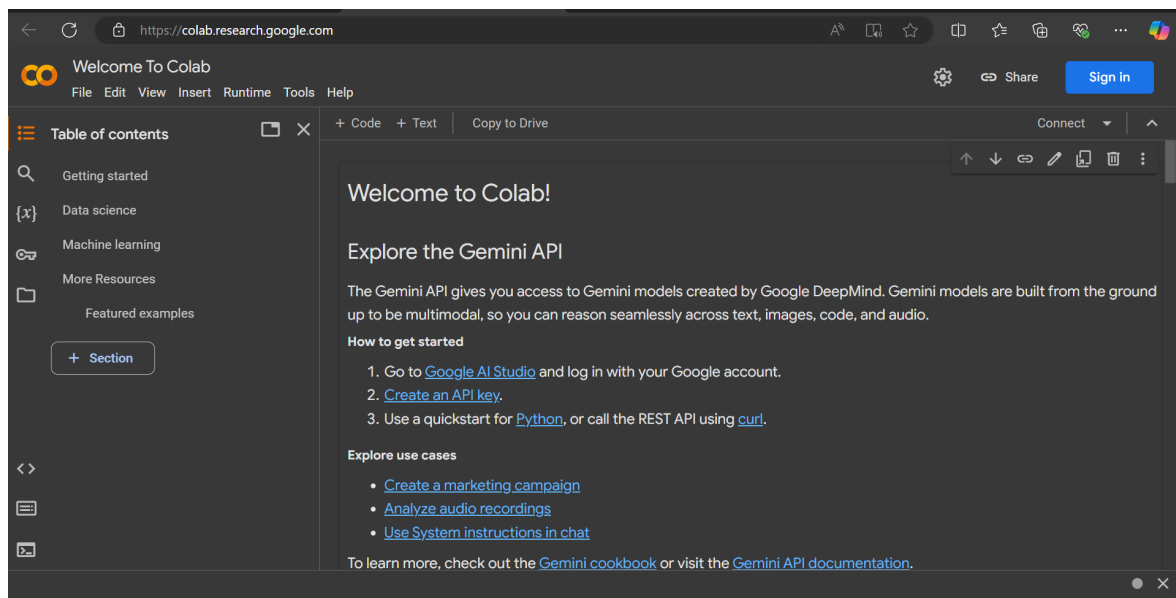
Mendaftar ke Google Colab

Gambaran umum

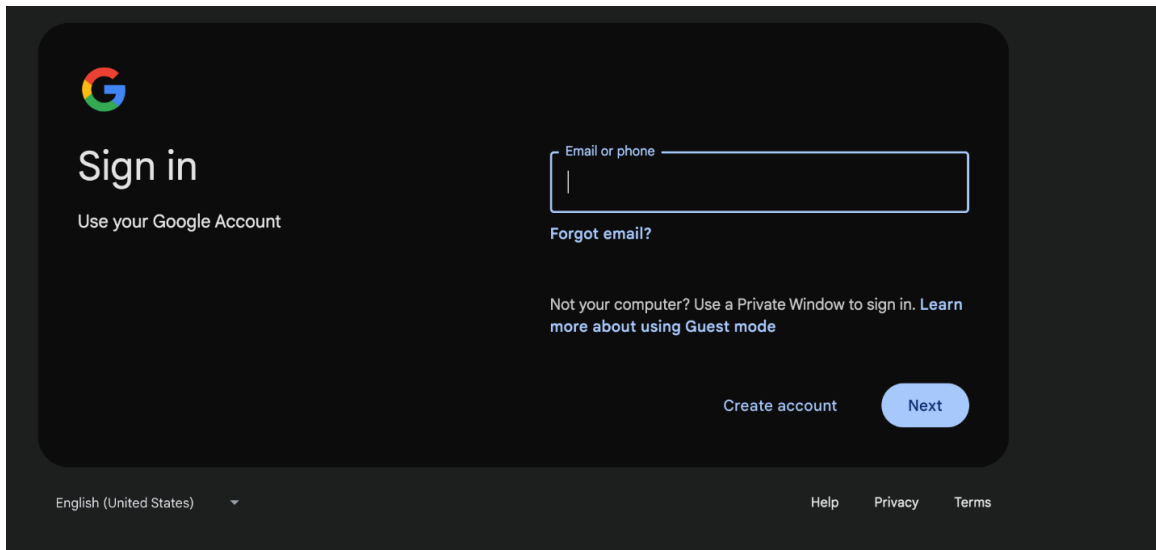
Pada prosedur ini, kamu akan menyiapkan akun Google Colab. Google Colab adalah platform cloud gratis yang memungkinkan kamu menjalankan kode di notebook, yang biasanya digunakan untuk pembelajaran mesin, ilmu data, dan tugas-tugas AI. Akun Google Colab memungkinkan kamu menginstal dan menggunakan alat yang diperlukan untuk menyelesaikan lab ini.

Instruksi

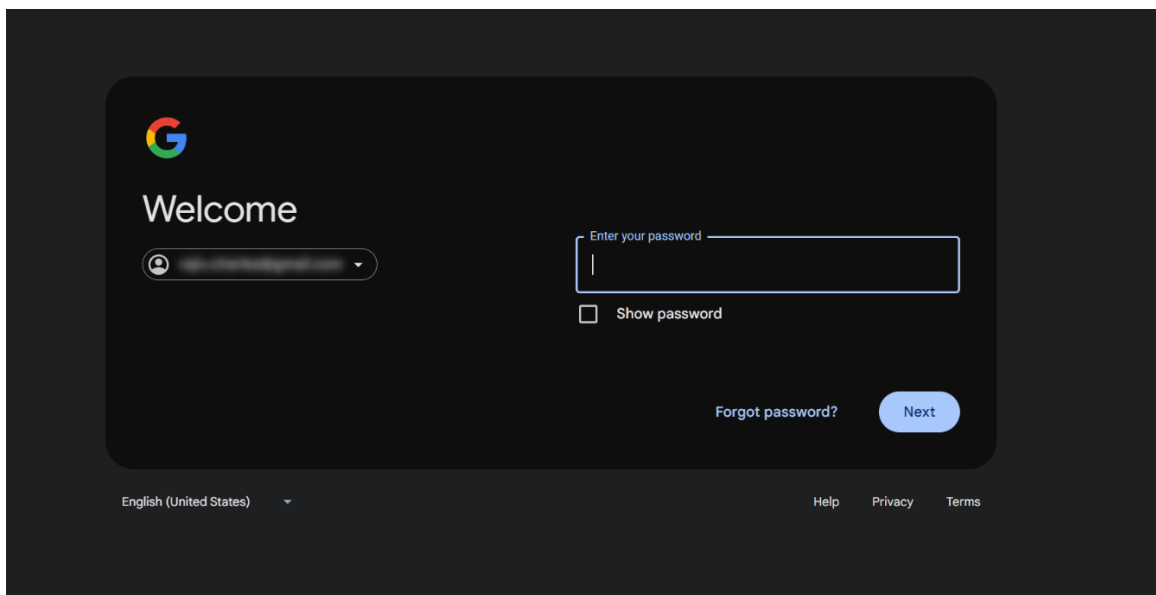
1. Untuk mendaftar, kunjungi situs web [Google Colab](https://colab.research.google.com).
2. Pilih **Sign in** (Masuk), seperti yang ditunjukkan pada screenshot berikut.



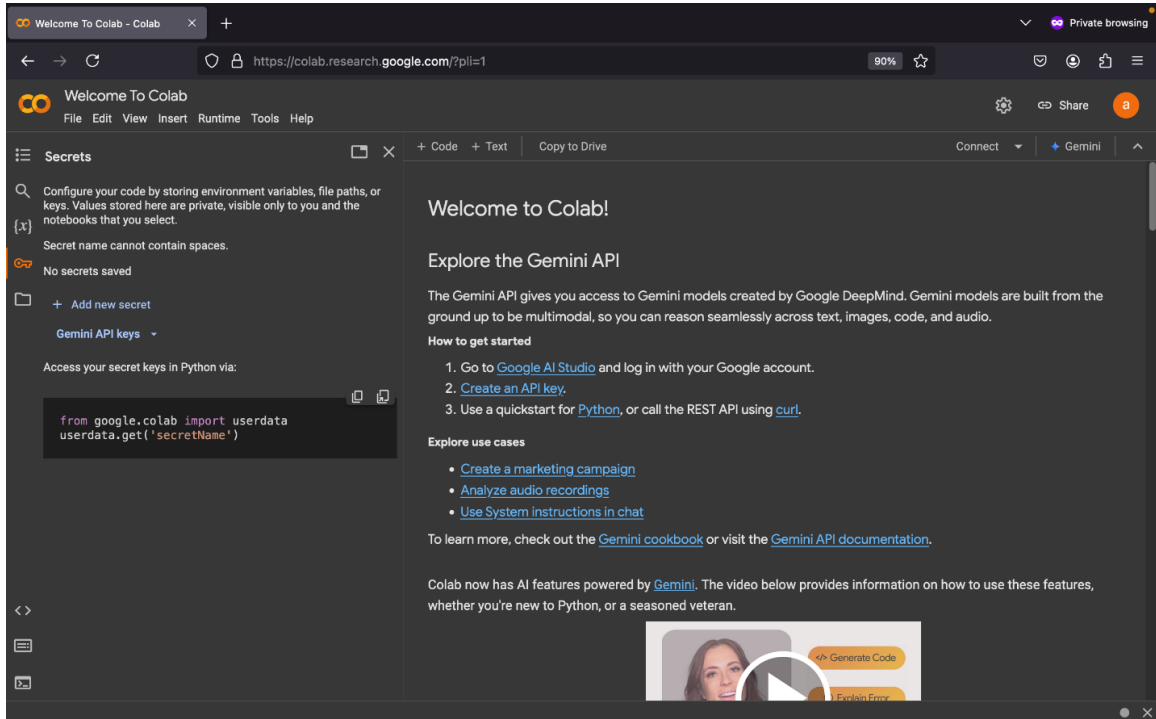
3. Masukkan email atau nomor ponsel kamu pada bidang **Email or phone** (Email atau ponsel), lalu pilih **Next** (Berikutnya).
Screenshot berikut menampilkan halaman masuk Google.



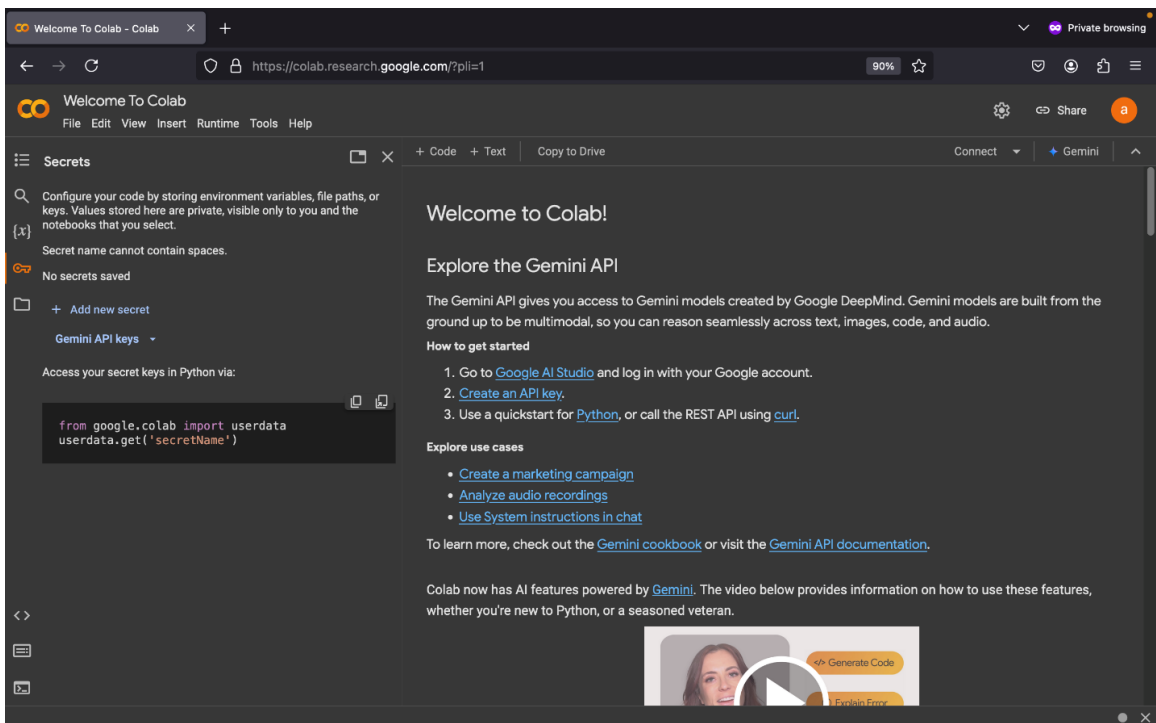
4. Masukkan kata sandi kamu di bidang **Enter your password** (Masukkan kata sandi), lalu pilih **Next** (Berikutnya).



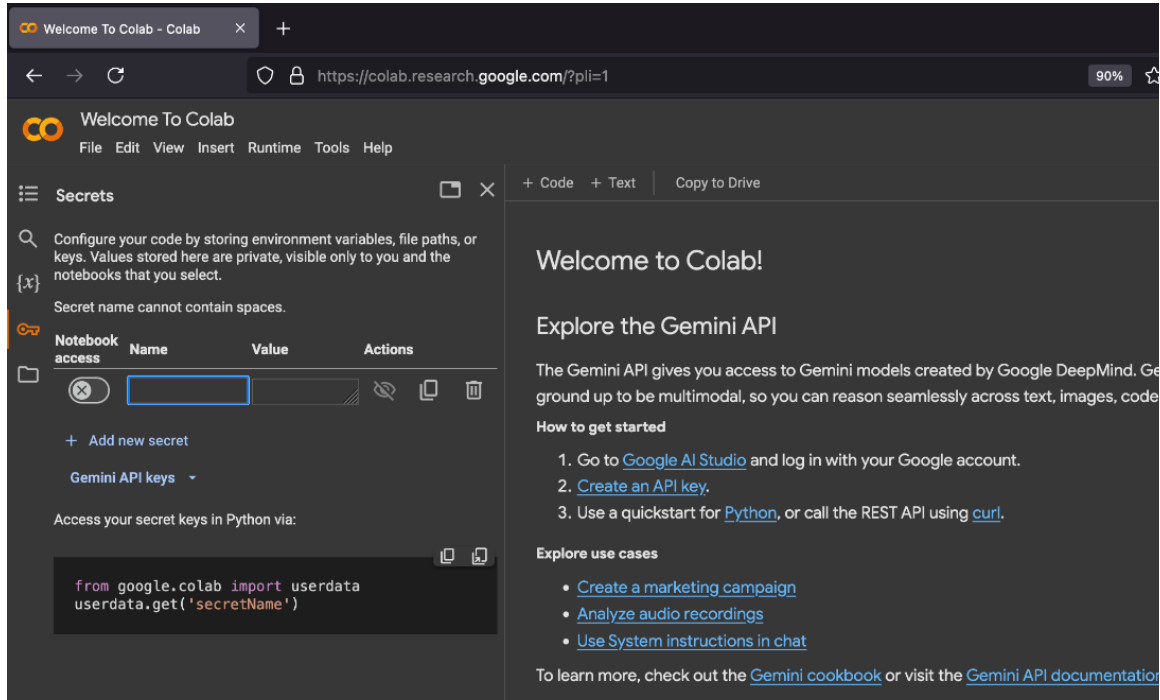
5. Untuk menyimpan token API Replicate kamu, pilih ikon kunci dari tab **Secrets** di halaman Welcome to Colab (Selamat Datang di Colab), seperti yang ditunjukkan pada screenshot berikut



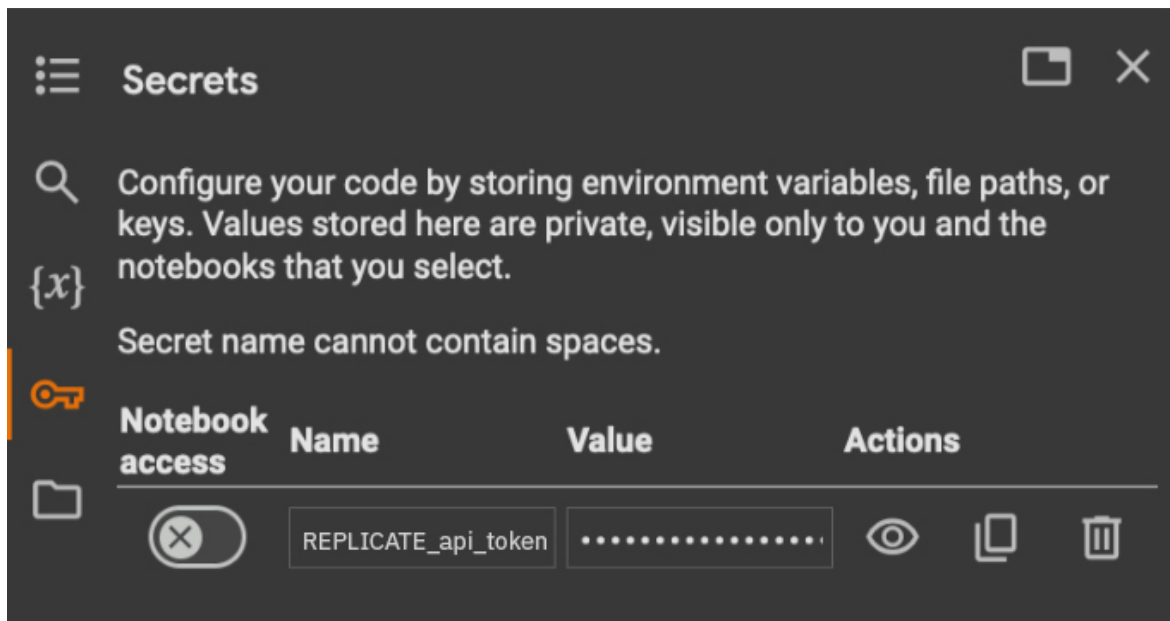
6. Pilih **Tambah** rahasia baru.



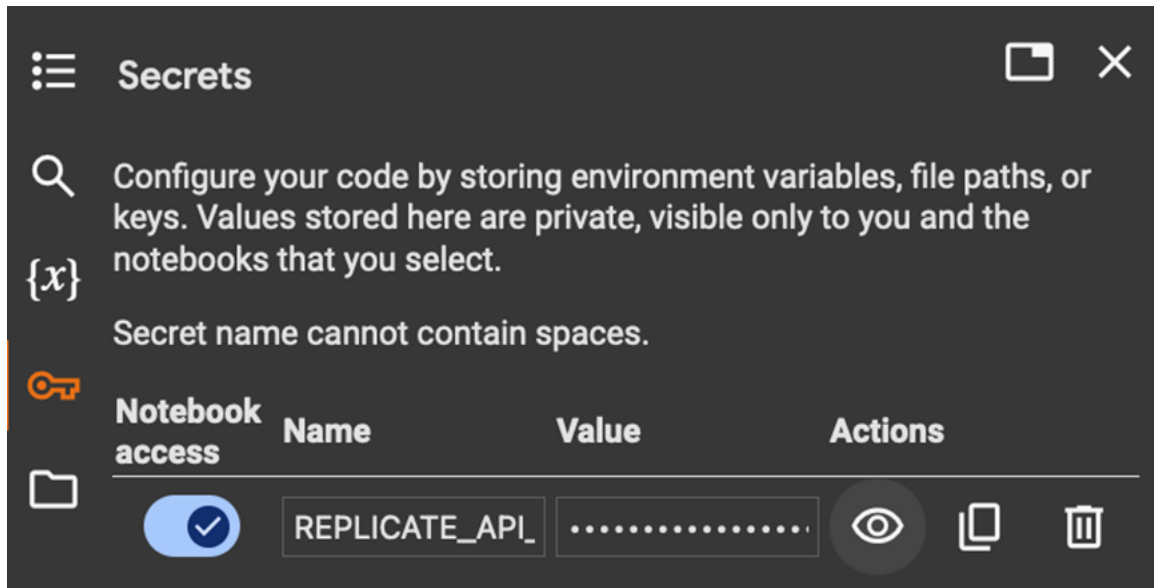
7. Ketik REPLICATE_api_token di bidang **Name** (Nama).



8. Tempelkan token API Replicate yang sudah kamu salin ke bidang **Value** (Nilai), seperti yang ditunjukkan pada screenshot berikut.



9. Aktifkan switch **Notebook access** (Akses Notebook) seperti yang ditunjukkan pada screenshot berikut. Lalu, pilih ikon keluar untuk keluar dari konfigurasi.



Menginisialisasi sgen AI dan alat

Gambaran umum

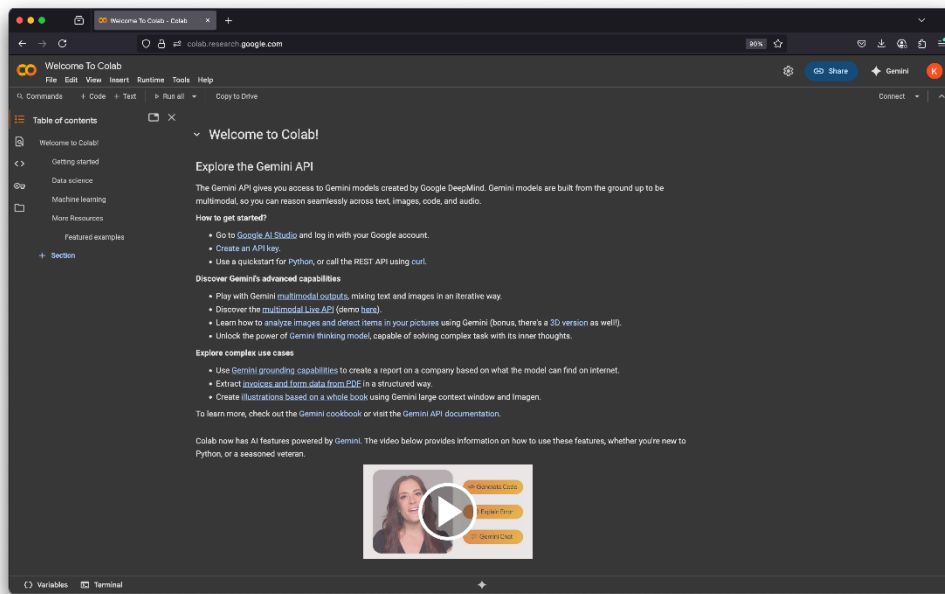
Dalam prosedur ini, kamu akan menginisialisasi agen AI TechMart beserta alat yang digunakannya dengan memuat Jupyter Notebook.

Dalam konteks agen AI, **alat** adalah fungsi yang membantu agen AI menjalankan tugas tertentu, seperti mencari informasi, memeriksa stok produk, atau melihat data. Pengguna memberikan prompt yang menentukan alat mana saja yang bisa diakses agen dengan menyertakannya ke dalam pengaturan agen. Saat merespons, agen akan mengirim kueri ke alat yang dipilih, menerima hasilnya, lalu menggunakan informasi itu untuk menghasilkan jawabannya.

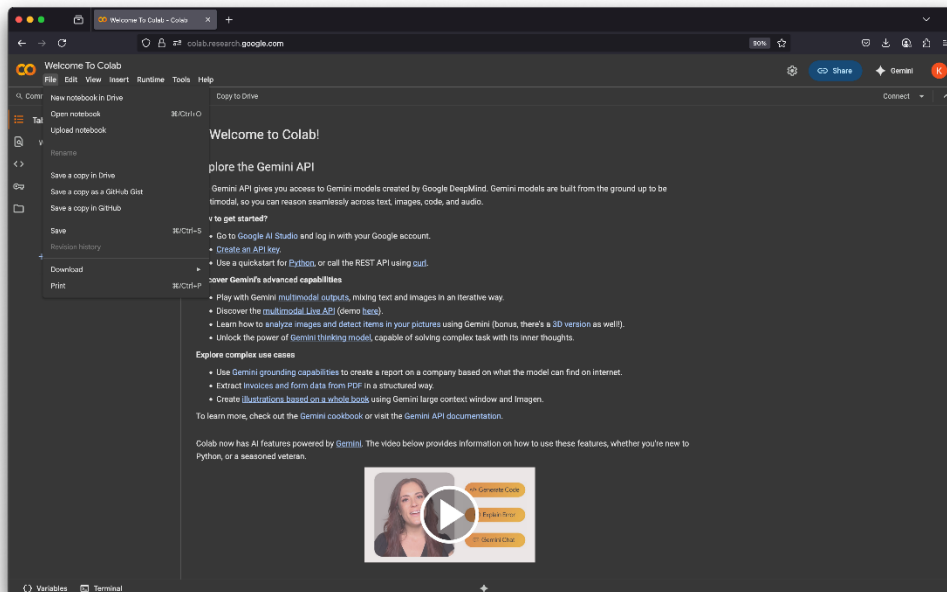
Jupyter notebook adalah dokumen yang bisa dibagikan yang menggabungkan kode komputer, deskripsi bahasa sederhana, data, dan visualisasi.

Instruksi

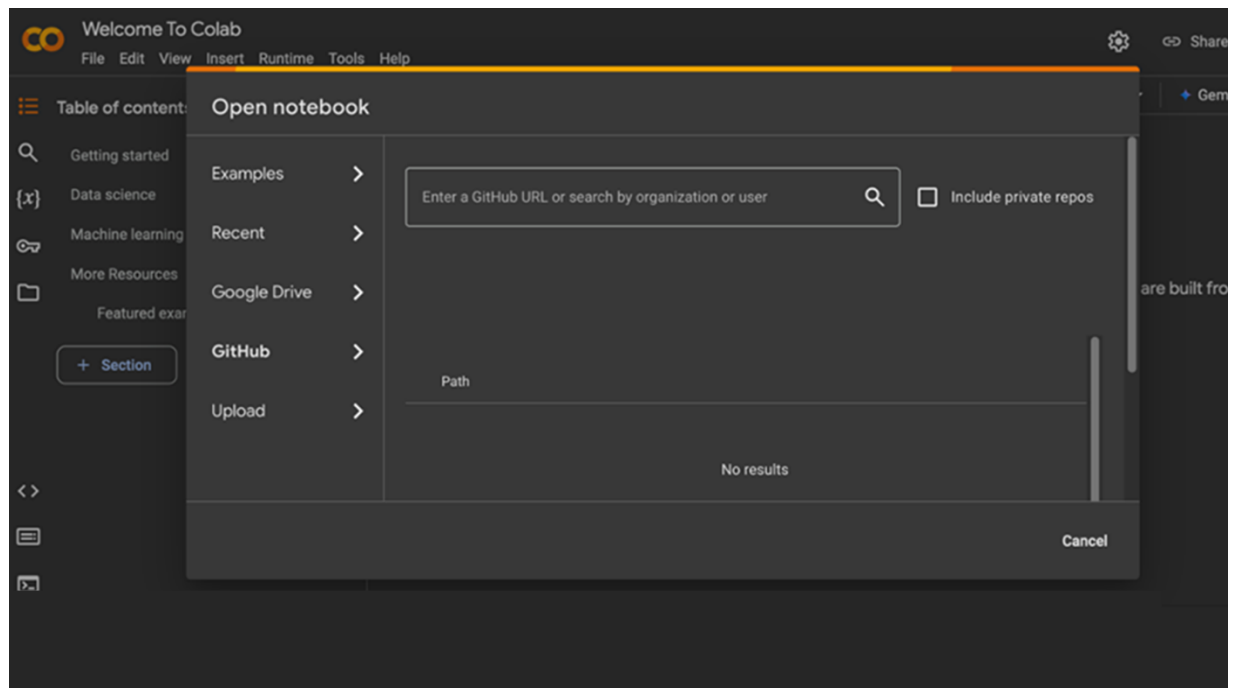
1. Pilih [Google Colab](#) untuk membuka Google Colab seperti yang ditunjukkan pada screenshot berikut. Untuk mulai bekerja di notebook, pilih **File** (File) dari menu



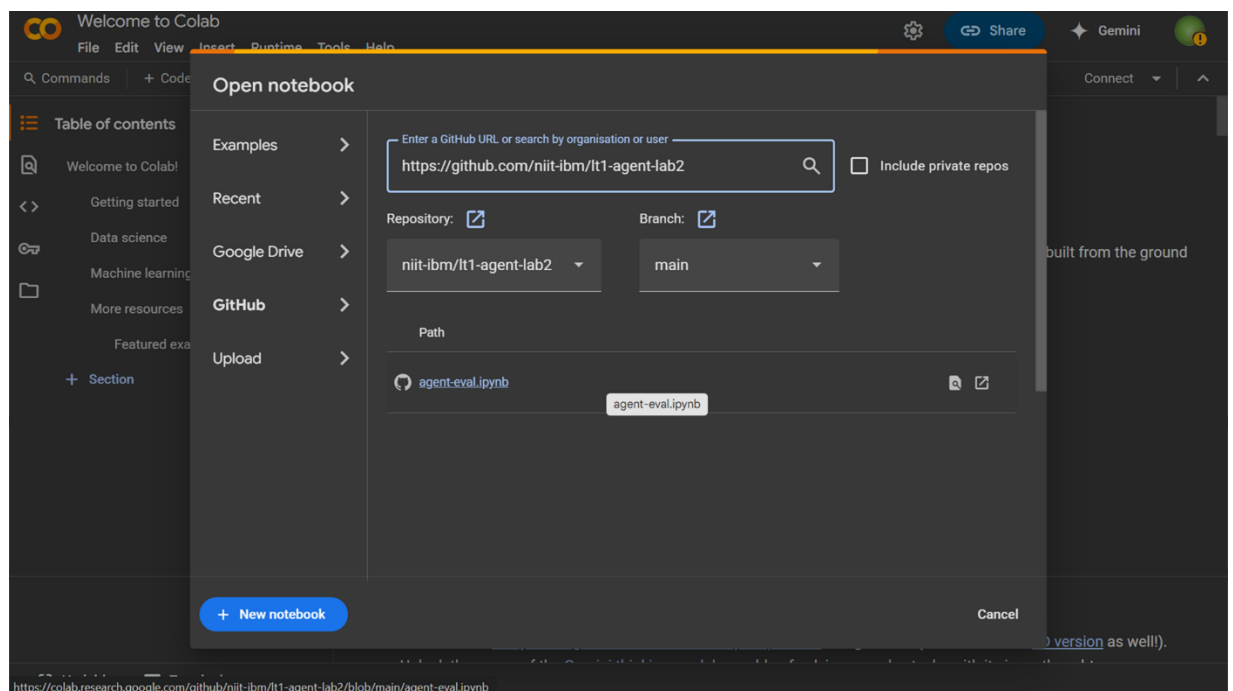
2. Pilih **Open notebook** (Buka notebook).



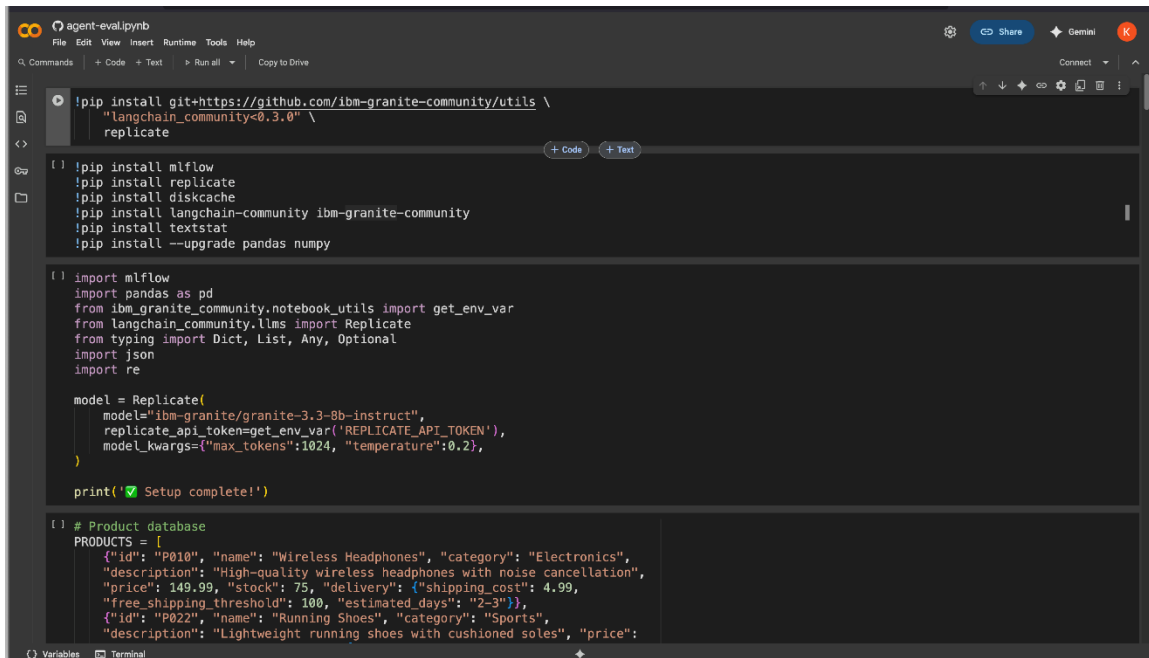
3. Dalam dialog "Open notebook (Buka buku catatan)", pilih **GitHub**. Untuk menemukan dan memuat Jupyter Notebook langsung di Google Colab, ketik uniform resource locator (URL) <https://github.com/niit-ibm/lt1-agent-lab2> di bidang **Enter a GitHub URL or search by organization or user** (Masukkan URL GitHub atau cari berdasarkan organisasi atau pengguna) Lalu, tekan Enter.



- Setelah kamu memasukkan URL, bidang **Repository** (Repositori) dan **Branch** (Cabang) akan terisi otomatis. Untuk membuka notebook, pilih **agent-eval.ipynb**.



Notebook agent-eval.ipynb terbuka di ruang kerja Colab, seperti yang ditunjukkan pada screenshot berikut.



```
!pip install git+https://github.com/ibm-granite-community/utls \
"langchain_community<0.3.0" \
replicate

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

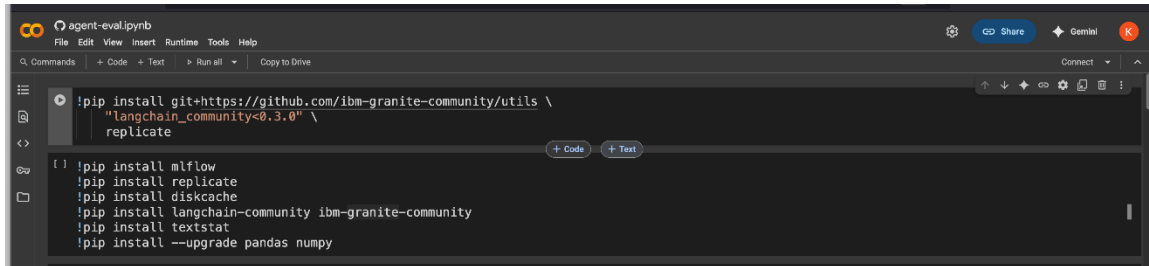
import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print('✅ Setup complete!')

# Product database
PRODUCTS = [
    {"id": "P010", "name": "Wireless Headphones", "category": "Electronics",
     "description": "High-quality wireless headphones with noise cancellation",
     "price": 149.99, "stock": 75, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 100, "estimated_days": "2-3"}},
    {"id": "P022", "name": "Running Shoes", "category": "Sports",
     "description": "Lightweight running shoes with cushioned soles", "price":
```

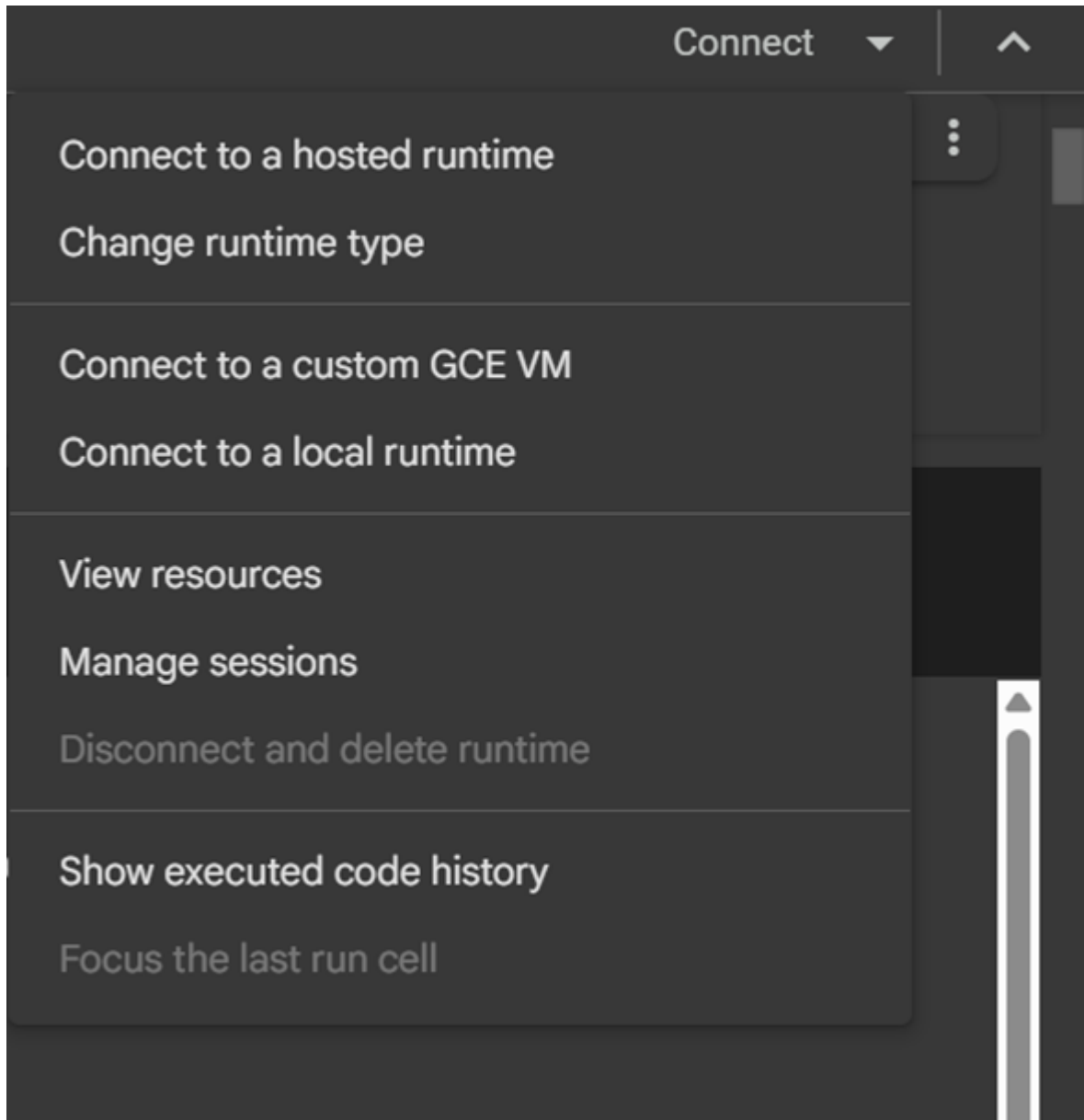
5. Untuk menjalankan kode, pilih panah drop-down **Connect** (Hubungkan).



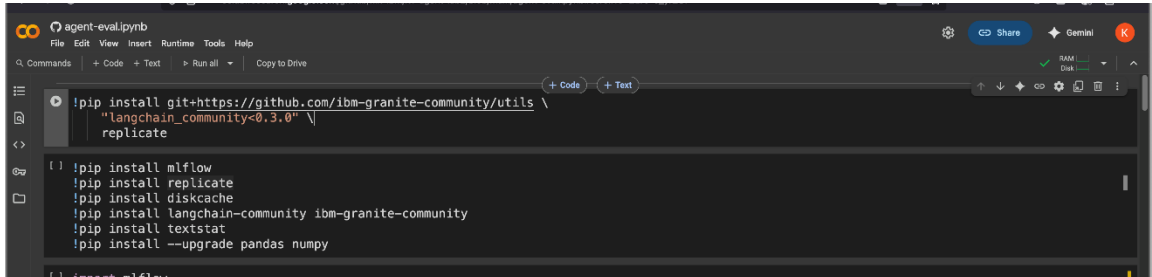
6. Dari daftar, pilih **Connect to a hosted runtime** (Hubungkan ke runtime yang di-host).

Screenshot berikut menampilkan Jupyter Notebook dengan opsi Connect to a hosted runtime (Hubungkan ke runtime yang di-host) di dalam daftar Connect.

Catatan: Saat kamu menggunakan Colab, kode tidak berjalan di mesin lokal kamu. Sebaliknya, kode dijalankan di lingkungan berbasis cloud yang disediakan Google, yang disebut **hosted runtime** (runtime yang di-host). Ini berarti tidak perlu menginstal Python atau pustaka apa pun di perangkatmu; semuanya sudah tersedia di lingkungan cloud.



Periksa toolbar **Colab** untuk memastikan ada tanda centang hijau, seperti yang ditunjukkan pada screenshot berikut. Ini menunjukkan bahwa koneksi berhasil, dan kode kamu siap dijalankan.

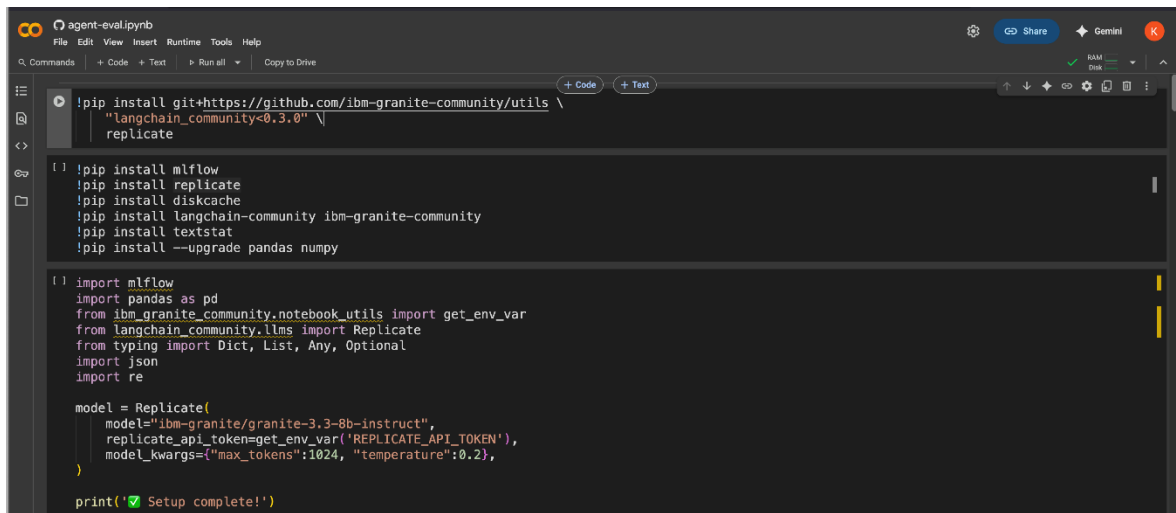


```
!pip install git+https://github.com/ibm-granite-community/utis \
"langchain_community<0.3.0" \
replicate

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy
```

- Agar agen AI kamu bisa bekerja dengan model IBM Granite dan menghasilkan respons yang bermanfaat, beberapa pustaka perlu diinstal. Pada sel pertama, pilih ikon **Run** (Jalankan) untuk memulai instalasi pustaka.

Catatan: Dalam Jupyter Notebook, setiap baris disebut **sel**. Sel-sel ini tidak diberi nomor tetapi diatur untuk dijalankan secara berurutan, dimulai dari sel pertama. Setiap sel kode mencakup ikon **Run** (Jalankan) (▶), yang kamu gunakan untuk mengeksekusi kode di dalam sel tersebut.



```
!pip install git+https://github.com/ibm-granite-community/utis \
"langchain_community<0.3.0" \
replicate

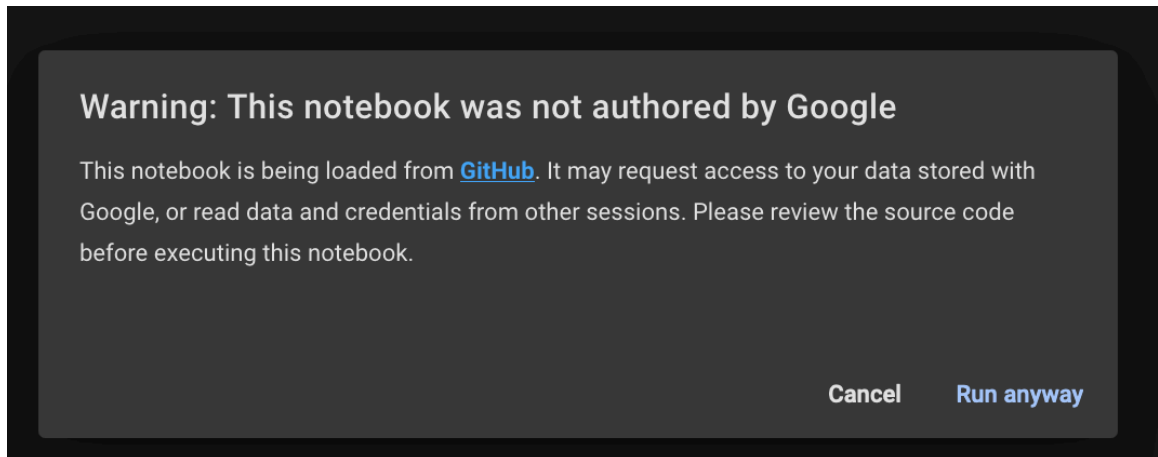
!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

!import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print('✅ Setup complete!')
```

- Pilih **Run anyway** (Terus jalankan) untuk melanjutkan pemuatan pustaka yang diperlukan.

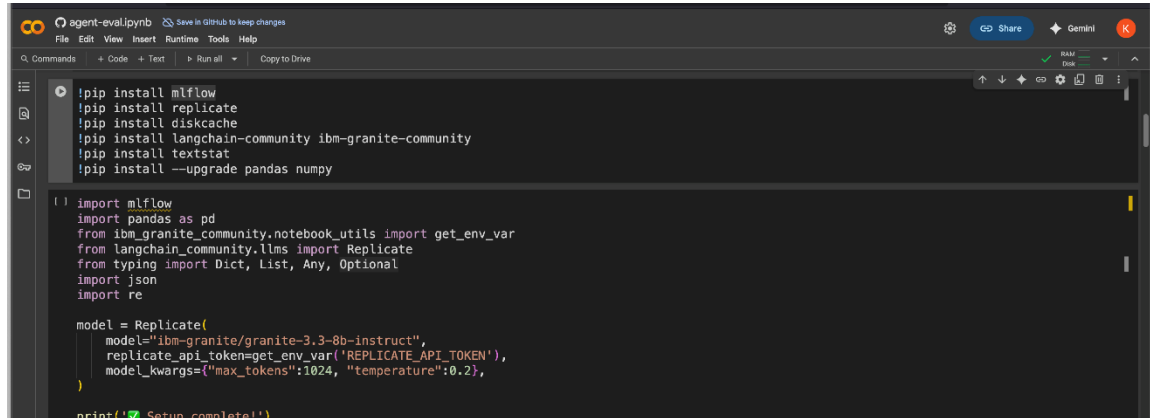


Tanda centang hijau akan muncul di sebelah ikon **Run** (Jalankan), yang menunjukkan bahwa pustaka yang diperlukan sudah berhasil diinstal.

```
!pip install git+https://github.com/ibm-granite-community/utils \
"langchain_community<0.3.0" \
.

Requirement already satisfied: typing_extensions==4.5.0 in /usr/local/lib/python3.11/dist-packages (from replicate) (4.14.0)
Requirement already satisfied: aiohappyhelli==2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (2.6.1)
Requirement already satisfied: aiosignal==1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (1.3.2)
Requirement already satisfied: attrs==17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (25.3.0)
Requirement already satisfied: frozenlist==1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (1.6.0)
Requirement already satisfied: multidict==4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (6.4.4)
Requirement already satisfied: propcache==0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (0.3.1)
Requirement already satisfied: yarl==1.7.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp==4.0.0->3.8.3->langchain_community<0.3.0) (1.20.0)
Collecting marshmallow==3.18.0 (from dataclasses-json==0.7.0->0.5.7->langchain_community<0.3.0)
  Downloading marshmallow-3.26.1-py3-none-any.whl.metadata (7.3 kB)
Collecting typing-inspect==0.9.0 (from dataclasses-json==0.7.0->0.5.7->langchain_community<0.3.0)
  Downloading typing_inspect-0.9.0-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (4.9.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (2025.4.26)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.11/dist-packages (from httpx==0.21.0->replicate) (3.10)
Requirement already satisfied: h11==0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx==0.21.0->replicate) (0.16.0)
Collecting langchain-text-splitters==0.3.0 (from langchain==0.3.0->0.2.17->langchain_community<0.3.0)
  Downloading langchain_text_splitters-0.2.4-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: jsonpatch==2.0.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core==0.3.0->0.2.43->langchain_community<0.3.0) (1.33)
Requirement already satisfied: orjson==4.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith==0.2.0->0.1.112->langchain_community<0.3.0) (3.10.18)
Requirement already satisfied: requests-toolbelt==0.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith==0.2.0->0.1.112->langchain_community<0.3.0) (1.0.0)
Requirement already satisfied: annotated-types==0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (2.33.2)
Requirement already satisfied: typing-inspection==0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic==1.10.7->replicate) (0.4.1)
Requirement already satisfied: charset-normalizer==4.0.0 in /usr/local/lib/python3.11/dist-packages (from requests==3.0.2->langchain_community<0.3.0) (3.4.2)
Requirement already satisfied: urllib3==3.0.0 in /usr/local/lib/python3.11/dist-packages (from requests==3.0.2->langchain_community<0.3.0) (2.4.0)
```

9. Untuk menginstal MLflow, pilih ikon **Run** (Jalankan) pada sel kedua, seperti yang ditunjukkan pada screenshot berikut.



```

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

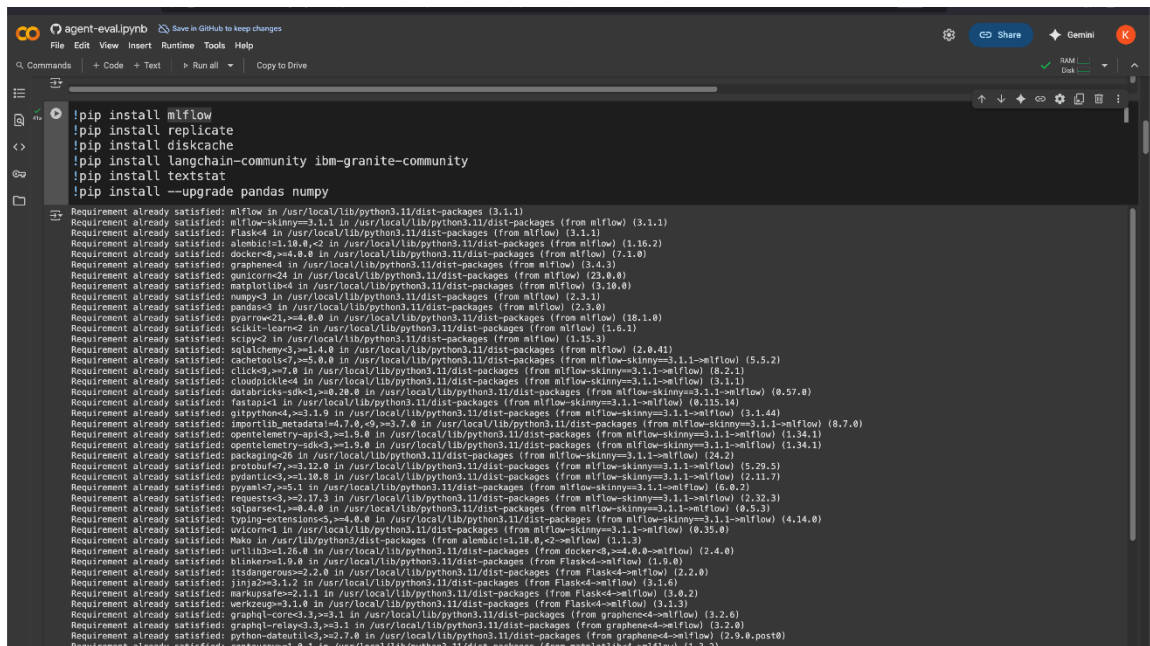
import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

model = Replicate(
    model="ibm-granite/granite-3.3-8b-instruct",
    replicate_api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens":1024, "temperature":0.2},
)

print(✅ Setup complete!)

```

Tanda centang hijau muncul di sebelah ikon **Run** (Jalankan), menandakan bahwa MLflow berhasil diinstal.



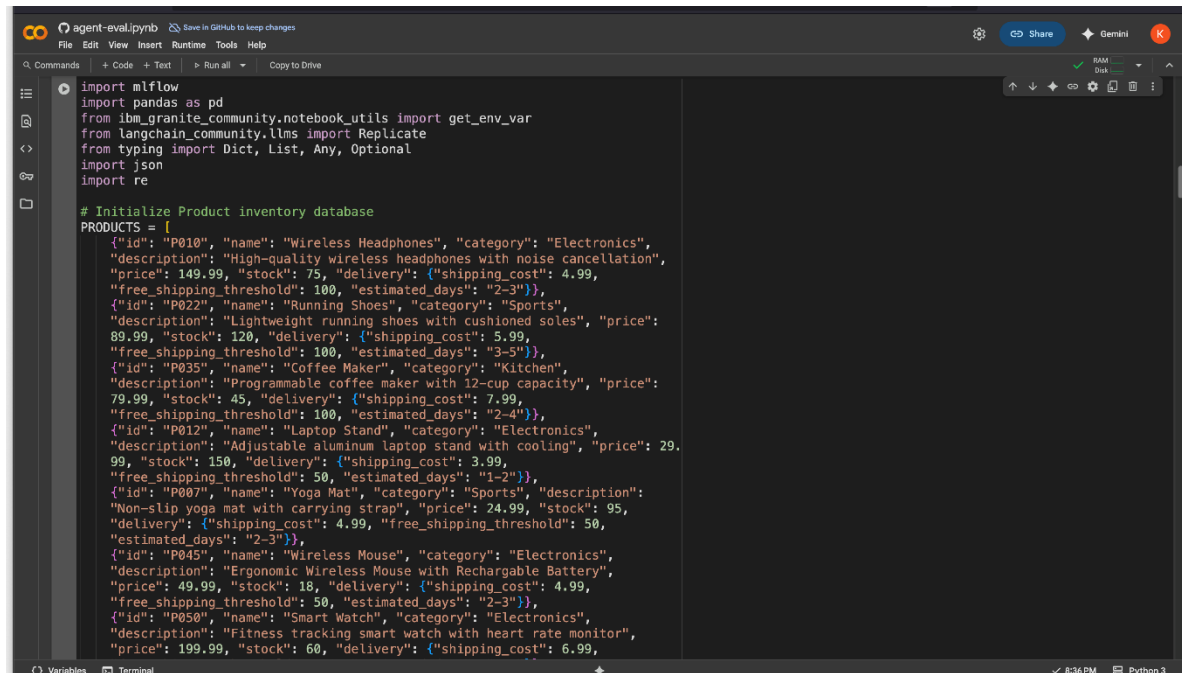
```

!pip install mlflow
!pip install replicate
!pip install diskcache
!pip install langchain-community ibm-granite-community
!pip install textstat
!pip install --upgrade pandas numpy

Requirement already satisfied: mlflow in /usr/local/lib/python3.11/dist-packages (3.1.1)
Requirement already satisfied: mlflow-skinny==3.1.1 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.1.1)
Requirement already satisfied: Flask<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.1.1)
Requirement already satisfied: alembic<1.10.0, >=1.9.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.16.2)
Requirement already satisfied: docker<6, >=4.0.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (7.1.0)
Requirement already satisfied: graphene<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.4.3)
Requirement already satisfied: gql-core<2.4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.3.0)
Requirement already satisfied: matplotlib<4 in /usr/local/lib/python3.11/dist-packages (from mlflow) (3.10.0)
Requirement already satisfied: numpy<3 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.3.1)
Requirement already satisfied: pandas<3 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.3.0)
Requirement already satisfied: pyarrow<21, >=4.0.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (18.1.0)
Requirement already satisfied: scikit-learn<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.6.1)
Requirement already satisfied: scipy<2 in /usr/local/lib/python3.11/dist-packages (from mlflow) (1.15.3)
Requirement already satisfied: sqlalchemy<3, >=1.4.0 in /usr/local/lib/python3.11/dist-packages (from mlflow) (2.0.41)
Requirement already satisfied: cachetools<7, >=5.0.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (5.5.2)
Requirement already satisfied: click<9, >=7.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (8.2.1)
Requirement already satisfied: cloudpickle<4 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (3.1.1)
Requirement already satisfied: databricks-sdk<1, >=0.28.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.57.0)
Requirement already satisfied: fastapi<1 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.115.10)
Requirement already satisfied: gptpython<4, <=3.1.9 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (3.1.44)
Requirement already satisfied: importlib_metadata<4.7.0, <=9, >=3.7.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (8.7.0)
Requirement already satisfied: opentelemetry-api<3, >=1.9.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (1.34.1)
Requirement already satisfied: opentelemetry-sdk<3, >=1.9.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (1.34.1)
Requirement already satisfied: packaging<26 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (24.2)
Requirement already satisfied: protobuf<7, >=2.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (5.29.5)
Requirement already satisfied: pydantic<3, >=1.10.8 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (2.11.7)
Requirement already satisfied: pyyaml<7, >=5.1 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (6.0.2)
Requirement already satisfied: requests<3, >=2.17.3 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (2.32.3)
Requirement already satisfied: sqlalchemy<3, >=0.4.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.5.3)
Requirement already satisfied: typing-extensions<5, >=4.0.0 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (4.14.0)
Requirement already satisfied: uvicorn<1 in /usr/local/lib/python3.11/dist-packages (from mlflow-skinny==3.1.1->mlflow) (0.35.0)
Requirement already satisfied: Make in /usr/lib/python3/dist-packages (from alembic<1.10.0, >=1.9.0->mlflow) (1.1.3)
Requirement already satisfied: urllib3<=1.26.0 in /usr/local/lib/python3.11/dist-packages (from docker<8, >=4.0.0->mlflow) (2.4.0)
Requirement already satisfied: blinker<1.9.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (1.9.0)
Requirement already satisfied: itsdangerous<=2.2.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (2.2.0)
Requirement already satisfied: Jinja2<=3.1.2 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.1.6)
Requirement already satisfied: MarkupSafe<=2.1.1 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.0.2)
Requirement already satisfied: Werkzeug<=3.1.0 in /usr/local/lib/python3.11/dist-packages (from Flask<4->mlflow) (3.1.3)
Requirement already satisfied: GraphQL-core<3.3, >=3.1 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (3.2.6)
Requirement already satisfied: GraphQL-relay<3, >=3.1 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (3.2.0)
Requirement already satisfied: GraphQL-dateutils<3, >=2.7.0 in /usr/local/lib/python3.11/dist-packages (from graphene<4->mlflow) (2.9.0.post0)

```

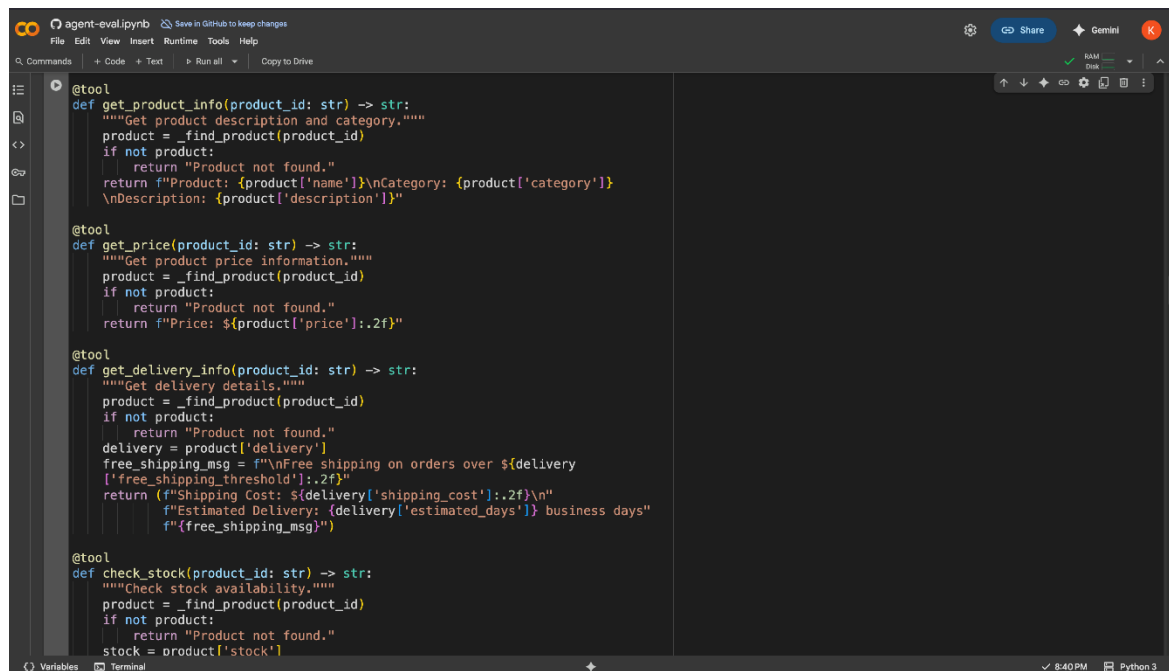
10. Untuk membuat basis data inventaris produk TechMart, pilih ikon **Run** (Jalankan) di sel pertama Tugas 2, seperti yang ditunjukkan pada screenshot berikut.



```
import mlflow
import pandas as pd
from ibm_granite_community.notebook_utils import get_env_var
from langchain_community.llms import Replicate
from typing import Dict, List, Any, Optional
import json
import re

# Initialize Product inventory database
PRODUCTS = [
    {"id": "P010", "name": "Wireless Headphones", "category": "Electronics",
     "description": "High-quality wireless headphones with noise cancellation",
     "price": 149.99, "stock": 75, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 100, "estimated_days": "2-3"}},
    {"id": "P022", "name": "Running Shoes", "category": "Sports",
     "description": "Lightweight running shoes with cushioned soles", "price":
     89.99, "stock": 120, "delivery": {"shipping_cost": 5.99,
     "free_shipping_threshold": 100, "estimated_days": "3-5"}},
    {"id": "P035", "name": "Coffee Maker", "category": "Kitchen",
     "description": "Programmable coffee maker with 12-cup capacity", "price":
     79.99, "stock": 45, "delivery": {"shipping_cost": 7.99,
     "free_shipping_threshold": 100, "estimated_days": "2-4"}},
    {"id": "P012", "name": "Laptop Stand", "category": "Electronics",
     "description": "Adjustable aluminum laptop stand with cooling", "price": 29.
     99, "stock": 150, "delivery": {"shipping_cost": 3.99,
     "free_shipping_threshold": 50, "estimated_days": "1-2"}},
    {"id": "P007", "name": "Yoga Mat", "category": "Sports", "description":
     "Non-slip yoga mat with carrying strap", "price": 24.99, "stock": 95,
     "delivery": {"shipping_cost": 4.99, "free_shipping_threshold": 50,
     "estimated_days": "2-3"}},
    {"id": "P045", "name": "Wireless Mouse", "category": "Electronics",
     "description": "Ergonomic Wireless Mouse with Rechargeable Battery",
     "price": 49.99, "stock": 18, "delivery": {"shipping_cost": 4.99,
     "free_shipping_threshold": 50, "estimated_days": "2-3"}},
    {"id": "P050", "name": "Smart Watch", "category": "Electronics",
     "description": "Fitness tracking smart watch with heart rate monitor",
     "price": 199.99, "stock": 60, "delivery": {"shipping_cost": 6.99,
```

11. Agar agen bisa memproses kueri pengguna dan menghasilkan respons yang bermakna, definisikan alat yang diperlukan seperti **get_product_info**, **get_price**, **get_delivery_info**, dan **check_stock**, seperti yang ditunjukkan pada screenshot berikut.



```
@tool
def get_product_info(product_id: str) -> str:
    """Get product description and category."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    return f"Product: {product['name']}\nCategory: {product['category']}\nDescription: {product['description']}"

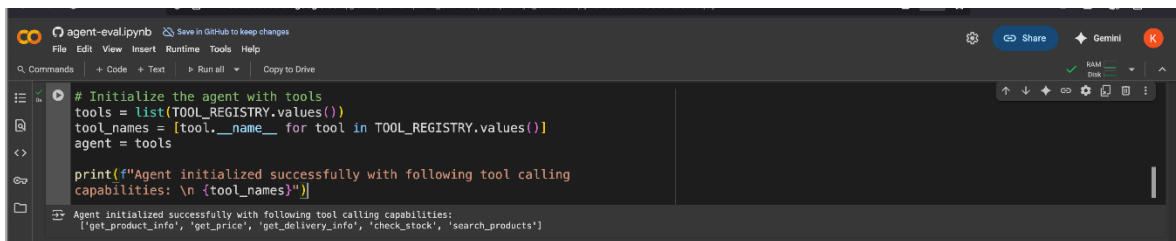
@tool
def get_price(product_id: str) -> str:
    """Get product price information."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    return f"Price: ${product['price']:.2f}"

@tool
def get_delivery_info(product_id: str) -> str:
    """Get delivery details."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    delivery = product['delivery']
    free_shipping_msg = f"\nFree shipping on orders over ${delivery['free_shipping_threshold']:.2f}"
    return (f"Shipping Cost: ${delivery['shipping_cost']:.2f}\n"
            f"Estimated Delivery: {delivery['estimated_days']} business days"
            f"{free_shipping_msg}")

@tool
def check_stock(product_id: str) -> str:
    """Check stock availability."""
    product = _find_product(product_id)
    if not product:
        return "Product not found."
    stock = product['stock']
```

12. Untuk menginisialisasi agen, pilih ikon **Run** (Jalankan) seperti yang ditunjukkan pada screenshot berikut.

Agen menganalisis input pengguna untuk mengetahui apakah yang diminta adalah deskripsi atau harga produk. Berdasarkan hal itu, agen akan menggunakan alat yang sesuai. Untuk jenis input lainnya, agen menghasilkan respons umum.



```
agent-eval.ipynb Save in GitHub to keep changes
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all Copy to Drive
# Initialize the agent with tools
tools = list(TOOL_REGISTRY.values())
tool_names = [tool.__name__ for tool in TOOL_REGISTRY.values()]
agent = tools

print(f"Agent initialized successfully with following tool calling
capabilities: \n {tool_names}")

Agent initialized successfully with following tool calling capabilities:
['get_product_info', 'get_price', 'get_delivery_info', 'check_stock', 'search_products']
```

Mengevaluasi trajektori dan respons akhir dari agen AI

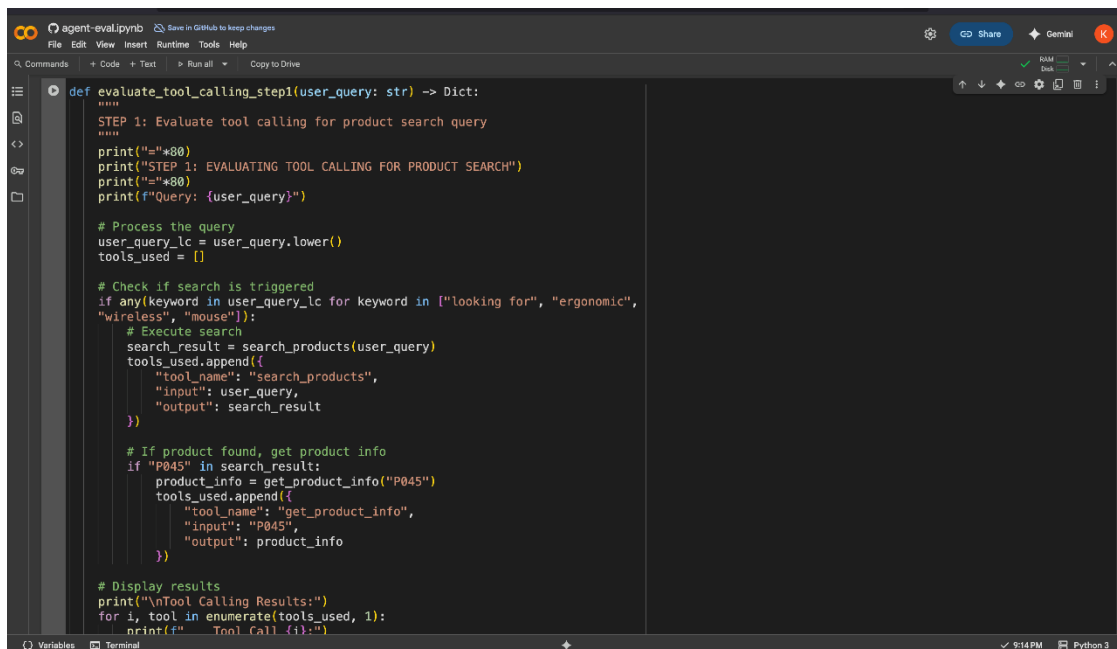
Gambaran umum

Dalam prosedur ini, kamu akan mengevaluasi trajektori agen dan menilai akurasi, kejelasan, serta relevansi respons akhirnya menggunakan MLflow.

Instruksi

1. Untuk menguji bagaimana agen merespons, pilih ikon **Run** (Jalankan) untuk memanggil fungsi **evaluasi**.

Catatan: Fungsi evaluasi ini merekam dan mencatat respons agen, membantu menentukan apakah agen sudah menganalisis kueri pengguna dengan benar dan memanggil alat yang sesuai. Fungsi ini memvalidasi apakah perilaku agen sudah selaras dengan penggunaan alat berbasis konteks dan pembuatan respons yang diharapkan.



```
def evaluate_tool_calling_step1(user_query: str) -> Dict:
    """
    STEP 1: Evaluate tool calling for product search query
    """
    print("="*80)
    print("STEP 1: EVALUATING TOOL CALLING FOR PRODUCT SEARCH")
    print("="*80)
    print(f"Query: {user_query}")

    # Process the query
    user_query_lc = user_query.lower()
    tools_used = []

    # Check if search is triggered
    if any(keyword in user_query_lc for keyword in ["looking for", "ergonomic",
    "wireless", "mouse"]):
        # Execute search
        search_result = search_products(user_query)
        tools_used.append({
            "tool_name": "search_products",
            "input": user_query,
            "output": search_result
        })

        # If product found, get product info
        if "P045" in search_result:
            product_info = get_product_info("P045")
            tools_used.append({
                "tool_name": "get_product_info",
                "input": "P045",
                "output": product_info
            })

    # Display results
    print("\nTool Calling Results:")
    for i, tool in enumerate(tools_used, 1):
        print(f"    Tool Call {i}:")
```

2. Untuk menguji bagaimana agen menginterpretasikan kueri pengguna dan menggunakan alat yang sesuai, pilih ikon **Run** (Jalankan) untuk mengeksekusi kode seperti yang ditunjukkan pada screenshot berikut.

```
"""Run the complete lab exercise in steps"""

print("MLflow Agent Evaluation Lab Exercise")
print("-"*80)

# Step 1: Tool calling evaluation for product search
query1 = "I'm looking for an ergonomic wireless mouse"
step1_result = evaluate_tool_calling_step1(query1)
```

3. Tinjau hasil pemanggilan alat untuk menentukan apakah agen sudah benar menginterpretasikan kueri pengguna, "I'm looking for an ergonomic wireless mouse (Aku mencari mouse nirkabel ergonomis)", dan memilih alat yang tepat untuk menghasilkan respons.

```
MLflow Agent Evaluation Lab Exercise
=====
STEP 1: EVALUATING TOOL CALLING FOR PRODUCT SEARCH
=====
Query: I'm looking for an ergonomic wireless mouse

Tool Calling Results:
Tool Call 1:
Tool Name : search_products
Input      : I'm looking for an ergonomic wireless mouse
Output     : Found the following products:
- Wireless Mouse (ID: P045): Ergonomic Wireless Mouse with Rechargeable Battery

Tool Call 2:
Tool Name : get_product_info
Input      : P045
Output     : Product: Wireless Mouse
Category: Electronics
Description: Ergonomic Wireless Mouse with Rechargeable Battery

Tool Calling Evaluation:
✓ Search tool called: Yes
✓ Product found: Yes
✓ Correct product (P045): Yes
```

4. Untuk mengevaluasi apakah agen bisa menangani kueri lanjutan dengan menggunakan beberapa alat sambil tetap mempertahankan pertanyaan dan konteks asli pengguna, pilih ikon **Run** (Jalankan) untuk mengeksekusi kode seperti yang ditunjukkan pada screenshot berikut.

```
Commands + Code + Text Run all Copy to Drive

# Step 2: Tool calling evaluation for stock query
query2 = "Is the product available in stock?"
step2_result = evaluate_tool_calling_step2(query2, "P045")
```

5. Tinjau output yang menunjukkan bagaimana agen menangani kueri, "Is the product available in stock? (Apakah produk ini tersedia dalam stok?)" Agen menggunakan alat seperti `get_product_info` dan `check_inventory` untuk menghasilkan respons berdasarkan kueri tersebut.

```
STEP 2: EVALUATING TOOL CALLING FOR STOCK QUERY
Follow-up Query: Is the product available in stock?
Product Context: P045

Tool Calling Results:
Tool Call 1:
Tool Name : check_stock
Input      : P045
Output     : Stock Status: Limited Stock (18 units available)

Tool Call 2:
Tool Name : get_product_info
Input      : P045
Output     : Product: Wireless Mouse
            Category: Electronics
            Description: Ergonomic Wireless Mouse with Rechargeable Battery

Stock Query Evaluation:
✓ Stock tool called: Yes
✓ Stock information provided: Yes
✓ Correct product context: Yes
```

6. Untuk mengevaluasi agen final, pilih ikon **Run** (Jalankan).

7. Untuk mengevaluasi agen final, pilih ikon **Run** (Jalankan).

```
# Step 3: Final Evaluation: Full MLflow evaluation
print("="*80)
print("STEP 3: RUNNING FULL MLFLOW EVALUATION")
print("="*80)

# Set up MLflow
mlflow.set_experiment("agent_evaluation_lab")

# Create evaluation data
response_eval = pd.DataFrame({
    "inputs": [
        "I'm looking for an ergonomic wireless mouse",
        "Is the product available in stock?",
        "Do you have ergonomic wireless mice in stock?"
    ],
    "targets": [
        "Product P045 is a wireless mouse with ergonomic support and programmable buttons",
        "Product P045 is currently available in your selected region",
        "Yes, Product P045 is available. It's an ergonomic wireless mouse with programmable buttons."
    ]
})

# Initialize and log the model
pyfunc_model = ReplicateToolCallingModel(
    model_name="ibm-granite/granite-3.3-8b-instruct",
    api_token=get_env_var('REPLICATE_API_TOKEN'),
    model_kwargs={"max_tokens": 1024, "temperature": 0.2}
)

# Log the model to MLflow
with mlflow.start_run() as run:
    mlflow.pyfunc.log_model(
        artifact_path="agent_model",
        python_model=pyfunc_model,
```

Setelah kode dieksekusi, ringkasan respons akan ditampilkan seperti yang ditunjukkan pada screenshot berikut. Perhatikan bahwa agen bisa menggunakan alat seperti `get_product_info` dan `check_inventory` dengan baik, serta mengambil informasi yang relevan sebagai respons atas kueri pengguna. Peserta kemudian harus meninjau hasil evaluasi MLflow, menganalisis kinerja pemanggilan alat, skor kualitas respons, serta metrik otomatis untuk menilai efektivitas agen dalam skenario e-commerce.

Catatan: Meski agen berhasil mengambil informasi relevan dengan alat, respons akhirnya tetap memerlukan evaluasi manusia untuk konfirmasi kebenarannya.

```
=====
Eval 1: Product Information
=====

Tool Calling Results:

Tool Call 1:
Tool Name : get_product_info
User Input : I'm looking for an ergonomic wireless mouse
Output : Product PM45 is a wireless mouse with ergonomic support and programmable buttons.

Tool Call 2:
Tool Name : check_inventory
User Input : Is Product PM45 available in stock?
Output : Product PM45 is currently available in your selected region.

=====
Final Agent Response Summary:
=====
Query : Do you have ergonomic wireless mice in stock?
Tool Used : check_inventory
Response : Yes, Product PM45 is available. It's an ergonomic wireless mouse with programmable buttons. Would you like to place an order?
Evaluation Score : 4/5
Feedback : The response is relevant and accurate, though it could be slightly more detailed about stock levels.
=====

🎉 Congratulations!

You've completed the simple AI agent evaluation lab! You've learned:
• How to work with a simple AI agent
• How to use different tools
• How to evaluate AI responses

Feel free to experiment with different queries and tools!
```

Kesimpulan

Selamat! Kamu berhasil mengevaluasi agen AI TechMart. Melalui pengujian terstruktur dan analisis dengan MLflow, kamu telah menilai efisiensi pemanggilan alat oleh agen serta kualitas respons finalnya, sehingga informasi produk bisa disampaikan dengan akurat, relevan, dan jelas.

© Hak Cipta IBM Corporation 2025.

Informasi yang terkandung dalam materi ini disediakan hanya untuk tujuan informasi dan disediakan SEBAGAIMANA ADANYA tanpa jaminan dalam bentuk apa pun, tersurat maupun tersirat. IBM tidak bertanggung jawab atas segala kerusakan apa pun yang timbul dari penggunaan, atau terkait dengan, materi ini. Tidak ada satu pun hal yang terkandung dalam materi-materi ini yang dimaksudkan untuk, atau akan berdampak pada, menciptakan jaminan atau pernyataan apa pun dari IBM atau para pemasok atau pemberi lisensinya, atau mengubah syarat dan ketentuan perjanjian lisensi yang berlaku yang mengatur penggunaan perangkat lunak IBM. Referensi dalam materi ini terhadap produk, program, atau layanan IBM tidak menyiratkan bahwa produk, program, atau layanan tersebut akan tersedia di semua negara tempat IBM beroperasi. Informasi ini didasarkan pada rencana dan strategi produk IBM saat ini, yang bisa diubah oleh IBM tanpa pemberitahuan. Tanggal rilis produk dan/atau kemampuan yang dirujuk dalam materi ini bisa berubah sewaktu-waktu atas kebijakan IBM sendiri berdasarkan peluang pasar atau faktor lainnya, dan tidak dimaksudkan sebagai komitmen terhadap ketersediaan produk atau fitur di masa mendatang dengan cara apa pun.

IBM, logo IBM, dan ibm.com adalah merek dagang dari International Business Machines Corp, yang terdaftar di banyak yurisdiksi di seluruh dunia. Nama produk dan layanan lainnya mungkin merupakan merek dagang dari IBM atau perusahaan lain. Daftar merek dagang IBM saat ini tersedia di Web di "informasi Hak hukum dan Merek Dagang" di www.ibm.com/hukum/copytrade.shtml.



Please Recycle
